Republic of Tunisia
*****
Ministry of Higher Education and Scientific Research
*****
University of Monastir
*****
Higher Institute of Computer Science and Mathematics of Monastir
Department of Computer Science

Order N°:

End of Studies Project Report

Presented in order to obtain the

National Bachelor's Degree in Computer Science

Specialization:

Software Engineering and Information Systems

By:

Computer Vision Models Fit for NPUs

Presented on in front of the jury composed of:

	President
	Member
Ms. Nada Haj Messaoud	Academic Supervisor
Mr Ilyes Tlili	Professional Supervisor

Academic Year: 2025 / 2026

Contents

List of Figures

List of Tables

List of Abbreviations

Table 1: Abbreviations used in the report

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BCE	Binary Cross Entropy
CIoU	Complete Intersection over Union
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CRISP-DM	Cross-Industry Standard Process for Data Mining
DFL	Distribution Focal Loss
EMA	Exponential Moving Average
FCN	Fully Convolutional Network
FLOPs	Floating Point Operations Per Second
FPN	Feature Pyramid Network
FPS	Frames Per Second
INT8	8-bit Integer (Quantization)
IoU	Intersection over Union
mAP	Mean Average Precision
NPU	Neural Processing Unit
ONNX	Open Neural Network Exchange
OOM	Out of Memory
PANet	Path Aggregation Network
R-CNN	Region-based Convolutional Neural Network
ROI	Region of Interest
RTSP	Real-Time Streaming Protocol
SKU	Stock Keeping Unit
SPP	Spatial Pyramid Pooling
SPPF	Spatial Pyramid Pooling Fast
SSD	Single Shot MultiBox Detector
VLM	Vision-Language Model
YOLO	You Only Look Once

General Introduction

Establishment

The rapid advancement of Artificial Intelligence, particularly in Computer Vision (CV), has transformed how industries monitor, secure, and optimize their operations. However, deploying these advanced models in real-world industrial environments often requires a strategic shift from cloud-centric processing to Edge-AI architectures. This shift is essential to ensure real-time responsiveness, minimize bandwidth dependency, and maintain strict data confidentiality.

This transition introduces significant engineering challenges. Neural Processing Units (NPUs)—specialized hardware accelerators designed for embedded edge devices—impose severe constraints on memory (SRAM), compute budgets, and latency. Deploying robust CV models under these conditions requires carefully balancing localization accuracy with real-time execution speeds, all while navigating the strict architectural requirements of the target hardware.

In this context, our project, developed within **in2-Technologies**, aims to design and implement an end-to-end workflow for producing and deploying optimized computer vision models for the **EYE-D** video analytics solution. The objective is to provide a smart, scalable, and highly efficient inference pipeline capable of executing advanced visual tasks directly on edge NPUs.

Our methodology combines rigorous data engineering with specialized single-stage architectures (such as the YOLO family) to deliver a robust and modular intelligent platform. The system is designed to address two primary operational domains: logistical optimization (forklift tracking and speed estimation) and industrial safety (real-time fire and smoke detection). Ultimately, this project enhances operational decision-making, ensures faster emergency responses, and improves overall safety compliance across industrial networks.

Work Organization

This report presents a comprehensive overview of the research, design, implementation, and optimization of the proposed Edge-AI computer vision pipelines. The structure of the report is as follows:

- **Chapter 1: Project Overview and Requirements Analysis**
  Introduces the project scope and the institutional context. Describes the motivation, objectives, specific NPU deployment constraints, and the CRISP-DM methodology adopted throughout the project.

- **Chapter 2: Phase 1 - Dataset Engineering & Lifecycle Strategy**
  Explores the foundational data engineering pipeline. This chapter details the data collection, curation, annotation, and augmentation processes, highlighting the critical role of dataset refinement in constrained edge deployments.

- **Chapter 3: Phase 2 - Industrial Safety Optimization**
  Details the modeling and deployment of fire and smoke detection systems. It focuses on architecture selection, handling challenging environmental artifacts to reduce false positives, and evaluating models under strict real-time constraints.

- **Chapter 4: Phase 3 - Logistical Monitoring & Speed Estimation**
  Presents the development of tracking and speed estimation pipelines for industrial vehicles (forklifts). It covers advanced visual tracking, homography transformations, and model optimization for real-world traffic management.

- **Chapter 5: Phase 4 - Zero-Shot Detection & Future Capabilities**
  Explores state-of-the-art multimodal AI architectures, including Vision-Language Models and Zero-Shot object detection. This chapter evaluates their potential and limitations for future integration into resource-constrained edge environments.

Chapter 1

PROJECT OVERVIEW AND REQUIREMENTS ANALYSIS

Introduction

This introductory chapter presents the scope of the end-of-studies project conducted within **in2-Technologies** in the context of the **EYE-D** solution. It introduces the hosting organization, clarifies the project context, and formalizes the problem statement and objectives. It also provides a high-level overview of the adopted approach, with particular emphasis on producing computer vision models that are compatible with deployment constraints on NPU-based systems.

1 Host Organization Presentation

1.1 in2-Technologies

in2-Technologies is an IT company that positions itself around digital innovation by delivering solutions that combine software engineering, creative design, and advanced technologies. In particular, the organization emphasizes applied Artificial Intelligence as a practical lever to deliver real-world value (Figure 1.1(a)).

According to the public communications provided for this report, in2-Technologies is actively involved in the innovation ecosystem through participation in events and technology programs. These activities help connect the company with partners and stakeholders, and they support continuous improvement by confronting products with real deployment needs.

In this context, in2-Technologies develops EYE-D, an AI-driven solution focused on video analytics and on-device intelligence.

1.2 EYE-D

EYE-D is an AI-driven solution developed within in2-Technologies and centered on computer vision and video analytics. It is presented as an approach that transforms existing video surveillance infrastructures into a system capable of producing actionable insights (Figure 1.1(b)).

Based on the provided communication materials, EYE-D is designed to integrate with existing camera networks and perform on-site processing. This positioning aims to reduce dependence on cloud processing, improve responsiveness, and address operational constraints such as privacy considerations and deployment cost.

EYE-D is also described as modular and expandable, enabling the activation or integration of AI modules according to the targeted needs. The communication materials highlight use cases related to security monitoring and industrial safety, with real-time alerting and reporting.

From an operational perspective, the provided materials emphasize the following aspects:

- **Integration**: EYE-D connects to an existing camera network and can be installed with minimal disruption.
- **On-device processing**: video analytics are executed locally to provide real-time insights and to reduce reliance on external connectivity.
- **Expandable modules**: the solution supports an evolving library of AI modules that can be enabled according to the deployment context.
- **Monitoring and reporting**: the solution provides alerts and reporting capabilities to support operational decision-making.

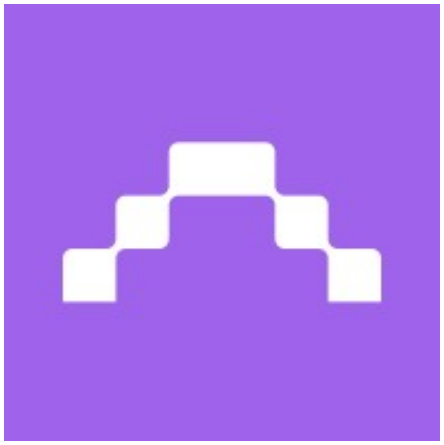
The brochure-style description provided for EYE-D points to a set of security and industrial safety features, including (non-exhaustively) access control and intrusion detection, people and vehicle detection, fire and emergency response related detection, and safety compliance monitoring. These capabilities motivate the need for robust, efficient, and deployable computer vision models under edge constraints, which is the focus of this project.

According to the public information provided for this report, EYE-D has been highlighted through multiple milestones and participations, including:

- Selection as a winner in an AI-focused innovation program (AI Garage Cohort 3 at Novation City).
- First place in the Tunisia IoT & AI Challenge 2025, followed by participation in the Arab IoT & AI Challenge in Dubai.
- Presence in technology events such as Big Tech Africa, STEP Dubai 2025, Security Expo London 2025, and GITEX Expand North Star.

As shown in Figure 1.1, the processing pipeline handles...

(a) in2-Technologies



(b) EYE-D

Figure 1.1: in2-Technologies and EYE-D logos

2 Project Presentation

2.1 Project Context

The project addresses the engineering challenge of developing and deploying computer vision models on resource-constrained platforms equipped with specialized accelerators. The target device and its detailed specifications are **confidential under a signed NDA**. In this report, the target platform is described in a general manner as an embedded system equipped with an **NPU (Neural Processing Unit)**.

Within the EYE-D context, the project contributes to an applied video analytics solution where on-device inference is required for responsiveness, operational constraints, and deployment practicality. The scope of the work includes a set of computer vision tasks required by the solution, covering multiple model families. The overall goal is not limited to training models, but extends to building complete and reliable pipelines that support deployment and monitoring in realistic operational conditions.

2.2 Problem Statement

Although computer vision models can achieve strong accuracy in research settings, deploying them on NPU-based platforms introduces several constraints and practical difficulties. In particular, the project must address the following issues:

- **Deployment constraints**: limited memory and compute budgets, strict latency requirements, and limited operator support depending on the target accelerator. Specifically, the target Neural Processing Units (NPUs) possess highly restricted localized SRAM (Static Random-Access Memory). Unlike cloud GPUs that rely on massive VRAM pools, edge NPUs require models to fit entirely within these small SRAM buffers to avoid severe latency penalties caused by fetching weights from slower off-chip DDR memory.

- **Model portability**: ensuring that trained models can be exported to an interoperable format (e.g., ONNX) and then converted into optimized inference artifacts.
- **Performance trade-offs**: balancing accuracy, inference speed, and power consumption when selecting architectures and optimization strategies.
- **Pipeline robustness**: handling runtime failures and edge cases through systematic error handling and stable orchestration across multiple vision tasks.

2.3 Requirements and Constraints

In line with the project context and problem statement, the work is driven by a set of requirements and constraints that guide design decisions and validation activities:

- **On-device inference**: the deployed solution must support local execution on the target embedded platform equipped with an NPU.
- **Latency and resource limits**: inference must remain compatible with strict latency targets and limited compute and memory budgets.
- **Deployment compatibility**: trained models must be exportable to an interoperable format and convertible into optimized inference artifacts.
- **Reliability in operation**: inference pipelines must handle runtime failures and edge cases through systematic validation and error handling.
- **Confidentiality constraints**: hardware specifications and internal data collection tooling remain confidential under NDA, requiring careful abstraction in documentation.

2.4 Project Purpose

The main objective of this project is to design an end-to-end workflow that produces **computer vision models fit for NPU deployment** while maintaining reliable inference pipelines for EYE-D.

The following guiding questions and objectives summarize the expected contribution of the project.

Table 1.1: Guiding questions and corresponding project objectives

Guiding Question	Project Objective
How can we deploy computer vision models under strict edge constraints?	Build an optimization and deployment pipeline compatible with NPU execution.
How can we maintain acceptable accuracy without exceeding latency and power budgets?	Select and tune suitable architectures, then benchmark trade-offs on representative workloads.
How can we ensure stable integration in a multi-task video analytics product?	Integrate inference into robust pipelines with monitoring, validation, and failure handling.

2.5 EYE-D Web Application

In addition to the on-device inference components, EYE-D is accompanied by a web-based interface that supports operational use. This interface is used to visualize analytics outputs, consult alerts and reports, and support supervision activities around deployed AI modules. In this report, the web application is considered as the user-facing component that consumes and presents the outputs produced by the embedded inference pipelines.

Objective Narrative

To clarify the scope of the project and align expectations, we define the core operational narrative. The project involves the development and integration of an end-to-end workflow for computer vision models within EYE-D, specifically targeting an embedded deployment platform equipped with an NPU (details confidential under NDA) within the in2-Technologies environment. This initiative is driven by the need to reduce reliance on cloud processing by enabling reliable on-device analytics, thereby improving responsiveness and meeting strict latency, compute, and power constraints. The project stakeholders include in2-Technologies, the EYE-D product team, and the operational users of the video analytics solutions, who benefit from continuous updates in response to evolving market needs.

3 State of the Art: Detection and Deployment Paradigms

This section establishes the technical foundations required to justify the architectural and methodological choices in the EYE-D project. The focus is strictly limited to NPU compatibility, neural network structure, and the mathematical framework enabling physical measurement.

3.1 Foundations of Computer Vision and Supervised Learning

In the context of industrial safety, Computer Vision fundamentally bifurcates into two distinct but sequential tasks: Object Detection, which localizes and classifies entities within a single

spatial frame, and Object Tracking, which associates these spatial detections across temporal sequences to maintain a persistent identity. To achieve the high reliability required for safety-critical environments, these systems rely on Supervised Learning. This paradigm mandates training the neural network on vast quantities of meticulously annotated data, ensuring the model learns deterministic, mathematically bounded representations of hazards like fire or moving forklifts rather than relying on unpredictable heuristics.

3.2 Evaluation Metrics and the COCO Dataset

The baseline capability of modern detection models is universally benchmarked against the COCO (Common Objects in Context) dataset [32], a large-scale collection of images spanning 80 diverse object categories. Performance on such datasets is quantified using Mean Average Precision (mAP), an evaluation metric that calculates the area under the Precision-Recall curve across multiple Intersection over Union (IoU) thresholds.

To formally evaluate model accuracy, the following standard metrics are defined:

- **Precision (P):** The ratio of true positive detections to the total number of positive predictions.
$$P = \frac{TP}{TP + FP} \tag{1.1}$$
- **Recall (R):** The ratio of true positive detections to the total number of actual ground truth objects.
$$R = \frac{TP}{TP + FN} \tag{1.2}$$
- **mAP@.5:** Mean Average Precision calculated at an Intersection over Union (IoU) threshold of 0.5.
- **mAP@.5:.95:** A stricter metric that averages mAP over multiple IoU thresholds (from 0.5 to 0.95 in steps of 0.05).

In edge deployments, mAP serves as the primary counterbalance to Frames Per Second (FPS), dictating the acceptable trade-off between localization accuracy and real-time execution speed.

3.3 YOLO as the Optimal Choice for Edge-AI and NPUs

In the context of Edge-AI, models must balance detection accuracy against strict latency and memory constraints imposed by Neural Processing Units (NPUs). Single-stage detectors like the YOLO (You Only Look Once) family are the industry standard for this task. They predict object categories and bounding box coordinates simultaneously in a single forward pass, avoiding the computational overhead of region-proposal networks found in two-stage models. This single-stage architecture sacrifices a marginal amount of peak accuracy in exchange for significantly

higher FPS and lower inference latency. Furthermore, the mature Ultralytics ecosystem allows YOLO models to be seamlessly exported to interoperable formats like ONNX and compiled into TensorRT engines, directly satisfying NPU deployment requirements.

3.4 Defining the Neural Network Backbone

In a convolutional neural network, the backbone functions as the primary feature-extracting engine, systematically downsampling raw input pixels into rich, multi-scale semantic representations. Architectures like CSPNet (Cross Stage Partial Network) [59] partition feature maps to reduce redundant gradient calculations, making the backbone lightweight and exceptionally efficient for real-time inference. For NPU deployments, optimizing this backbone is the most vital step in preventing computational bottlenecks and maintaining real-time execution at the edge.

3.5 Tooling for Data Engineering: Label Studio and Roboflow

The accuracy of supervised learning models is inextricably linked to the quality of their underlying datasets. Modern data engineering relies on specialized tooling to manage the dataset lifecycle. **Label Studio** [72] is widely adopted for computer-assisted annotation, providing a flexible interface for human annotators to correct pre-generated bounding boxes, significantly accelerating the labeling process. Conversely, **Roboflow** [73] serves as the industry standard for dataset versioning, augmentation, and analytics, allowing engineers to track class distributions and dataset health over time.

Figure 1.2: Label Studio

Figure 1.3: Roboflow

3.6 The YOLO Annotation Format

To ensure interoperability between annotation tools and training pipelines, bounding box data must be standardized. The YOLO Annotation Format represents each object as a single line in a text file:

$$< class\_id > < x\_center > < y\_center > < width > < height > \tag{1.3}$$

A critical feature of this format is that all spatial coordinates are normalized as percentages of the image dimensions (values between 0.0 and 1.0). This normalization is essential for robust machine learning operations; it allows the neural network to dynamically handle and resize images of vastly different resolutions during training and inference without the need to mathematically recalculate or re-label the bounding boxes.

4 Existing Solutions

Several solution strategies are commonly adopted to deliver computer vision capabilities under operational constraints:

- **Cloud-centric processing**: cameras stream data to a server or cloud environment that runs inference, then returns results to client applications.
- **Edge computing with accelerators**: inference is executed close to the cameras on embedded devices equipped with GPUs or NPUs, reducing latency and dependency on connectivity.
- **Standardized model exchange and deployment tooling**: workflows often rely on interoperable formats (e.g., ONNX [42]) and on optimization toolchains (quantization, pruning, operator fusion) to obtain deployable artifacts.
- **Modular analytics pipelines**: products typically combine multiple specialized models (detection, tracking, classification) orchestrated to produce higher-level events.

5 Critique of Existing Solutions

Although these approaches are effective in many contexts, they present limitations that motivate the focus of this project:

- **Latency and bandwidth trade-offs**: cloud-centric approaches increase dependency on network connectivity and may not satisfy strict real-time constraints.
- **Conversion and portability issues**: model export and optimization may fail due to unsupported operators, numerical differences, or vendor-specific constraints.
- **Accuracy degradation**: deployment-oriented optimizations such as quantization may reduce accuracy if not carefully validated and calibrated.
- **Operational robustness**: multi-model pipelines can be sensitive to runtime failures and edge cases, requiring systematic monitoring and error handling.
- **Vendor lock-in risks**: NPU deployment toolchains may impose specific constraints that limit portability across platforms.

6 Methodology & Evaluation Metrics

This project uses the CRISP-DM (Cross-Industry Standard Process for Data Mining) technical framework to formally capture the engineering maturity of the solution and deliver deployable computer vision components for EYE-D.

6.1 Evaluation Metrics Definition

Before analyzing the quantitative performance of the implemented models in the subsequent chapters, it is crucial to mathematically define the standard metrics from the COCO evaluation framework utilized to assess the quality of the object detection models:

- **Precision (P):** The ratio of true positive detections to the total number of positive predictions. This metric indicates how well the model avoids false alarms (e.g., misclassifying a light as fire).
- **Recall (R):** The ratio of true positive detections to the total number of actual ground truth objects. This metric indicates the model's ability to find all instances of the target object.
- **mAP@.5:** Mean Average Precision calculated at an Intersection over Union (IoU) threshold of 0.5.
- **mAP@.5:.95:** A stricter metric that averages mAP over multiple IoU thresholds (from 0.5 to 0.95 in steps of 0.05), rewarding highly accurate bounding box localization.

6.2 Project Steps

Both the logistical optimization phase (Forklift Detection and Speed Estimation) and the industrial safety phase (Fire and Smoke Detection) cycled through the following six CRISP-DM steps:

- **1. Business Understanding:** We defined the core operational needs for the EYE-D system. For the logistical optimization phase, the objective was optimizing warehouse logistics and traffic management via Forklift tracking and speed estimation. For the safety phase, the focus shifted to industrial safety, requiring real-time fire and smoke detection to trigger immediate emergency alerts.
- **2. Data Understanding:** High-quality data acquisition was critical. The first phase utilized a custom, curated Forklift dataset to address the absence of industrial vehicles in standard COCO datasets. The safety phase leveraged the HuggingFace `fire-smoke-hardnegatives-int8` dataset, emphasizing the need to understand environmental hazards and challenging lighting conditions.

- **3. Data Preparation:** Advanced data augmentations were applied to ensure generalization in real-world industrial environments. For both phases, techniques such as Mosaic and Mixup were integrated. Crucially, the safety phase incorporated hard-negative mining to actively reduce false positives (e.g., misclassifying steam or reflections as fire).
- **4. Modeling:** Architecture selection was strictly guided by NPU hardware constraints. For the first phase, YOLOv5 was chosen for its proven low-latency NPU optimization. For the safety phase, YOLOv8n was selected to leverage its advanced multi-scale feature extraction while remaining compact enough for edge deployment.
- **5. Evaluation:** Models were evaluated quantitatively using industry-standard metrics. We balanced Mean Average Precision (mAP) for detection accuracy against Frames Per Second (FPS) to validate real-time execution capabilities on the target NPU platform.
- **6. Deployment:** The final PyTorch models from both phases were exported to ONNX format, ensuring cross-platform interoperability. These ONNX artifacts were then compiled into optimized inference engines and deployed directly onto the edge NPU, fulfilling the project's strict latency and privacy requirements.

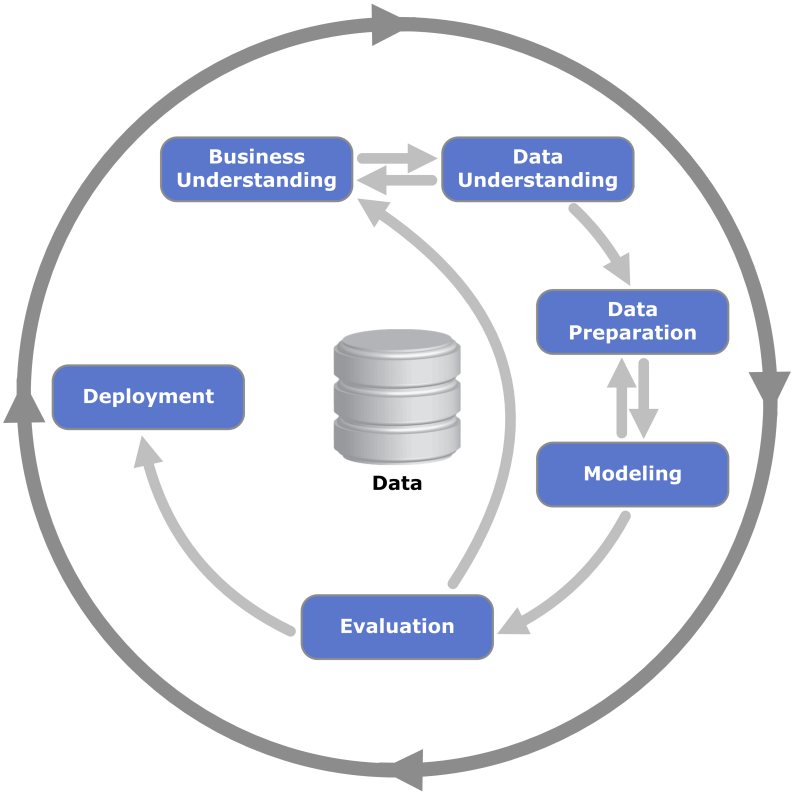


Figure 1.4: CRISP-DM lifecycle governing the project methodology and technical execution.

As illustrated in Figure 1.4, the project methodology is governed by a circular CRISP-DM lifecycle that ensures rigorous technical execution across all steps. This framework specifically accommodates an iterative feedback loop between the evaluation and data preparation stages.

Such a loop was essential for implementing Hard-Negative Mining in the Fire/Smoke dataset, allowing the system to continuously refine its understanding of challenging environmental artifacts and reduce false positive alerts.

For project tracking and collaboration, **GitHub** was used as the central platform, supporting version control and coordination of development activities.

7 Chapter Summary

This chapter presented the hosting organization (in2-Technologies) and the context of the EYE-D solution, then formalized the project overview, requirements, objectives, and the adopted project management approach. The next two chapters present the two implementation phases with their integrated state of the art studies, design diagrams, and implementation details under NPU deployment constraints. While general detection models provide a baseline, the specific safety requirements of industrial forklift tracking necessitate a transition toward real-time optimized architectures. Every algorithmic decision going forward is strictly evaluated against the limited SRAM and thermal thresholds of the target hardware. The following chapter details the selection and optimization of these models for NPU deployment.

Chapter 2

Phase 1 - Dataset Engineering & Lifecycle Strategy

1 Chapter Introduction

This chapter covers the critical "Data Understanding" and "Data Preparation" phases of the project, structured according to the CRISP-DM methodology [1]. A core premise of this research is that model performance on industrial Neural Processing Units (NPUs) is fundamentally limited by data quality, not merely by the underlying network architecture. To achieve reliable deployment, the dataset must be meticulously engineered to meet strict hardware and accuracy constraints.

1.1 CRISP-DM Methodology Mapping

This phase is strictly aligned with the Cross-Industry Standard Process for Data Mining (CRISP-DM) framework. The "Data Understanding" phase involved an initial exploratory analysis of the warehouse environment, identifying the types of cameras available, their respective blind spots, and the typical frequency of safety incidents. We documented the baseline operational metrics, noting that on an average day, a warehouse might see hundreds of forklift-pedestrian interactions but zero actual fires. This extreme rarity of critical events heavily influenced our data collection strategy.

The subsequent "Data Preparation" phase, which forms the bulk of this chapter, focuses on transforming this raw understanding into a structured tensor format suitable for deep learning. This involved not only labeling the data but also designing a robust data pipeline that could handle the continuous influx of new footage. The CRISP-DM methodology emphasizes that data preparation is rarely a single step; it is an iterative process. As we transitioned into the modeling phase (Chapter 3), we frequently returned to this preparation phase to refine our labels based on the model's false-positive analyses.

2 Raw Data Collection

The data lifecycle began with raw data acquisition directly from the target industrial environment, utilizing a network of specialized industrial sensors and CCTV feeds. The initial state of the collected footage was unstructured and characterized by high informational entropy. It contained a wide variety of "real-world" labels capturing the chaotic dynamics of warehouse operations, personnel movement, and environmental noise, all recorded prior to any structured optimization.

To ensure the dataset accurately reflected the deployment environment, data was sampled across multiple shifts, varying lighting conditions (e.g., bright daylight through skylights, artificial night-time LED illumination), and diverse operational scenarios. High-definition 4K cameras were strategically positioned at operational choke points such as loading docks, main storage aisles, and blind intersections where forklift-to-pedestrian interactions are most frequent and dangerous.

A critical challenge during this phase was managing the sheer volume of redundant temporal data. Continuous video feeds generated terabytes of footage where nothing of interest occurred. To mitigate this, a preliminary frame-extraction script was deployed to sample frames dynamically. During high-activity periods (e.g., active loading), frames were extracted at 2 frames per second (FPS), whereas during idle periods, the extraction rate dropped to 1 frame every 10 seconds. This intelligent sampling strategy drastically reduced storage overhead, eliminated sequential frame bias, and maximized the informational density of the raw dataset.

3 The Pipeline Loop

As illustrated in Figure 2.1, the project adopts a Data-Centric AI approach, where the focus remains on the iterative refinement of the dataset rather than static model training. Rather than following a strict linear progression, the data workflow was designed to harness the synergy between automated systems and human expertise.

Continuous Ingestion: Unlike traditional machine learning pipelines that rely on a static, one-time data upload, raw data is continuously fed into the system from the left side of the workflow. This ensures the model constantly adapts to the evolving reality of the warehouse floor.

The "Human-in-the-Loop" (HITL) Element: The pipeline integrates a semi-automated labeling loop, significantly reducing the temporal overhead of manual annotation while maintaining the high precision required for industrial safety applications. Utilizing Label Studio as the core annotation engine, this step creates a "Gold Standard" dataset where an AI model proposes initial labels, and the human engineer verifies and refines them.

The Feedback Mechanism: The circular arrows in the diagram represent a powerful acceleration mechanism. As the model's automated detection capabilities improve, it is

increasingly trusted to auto-label incoming raw data. This symbiotic relationship creates a progressively faster development cycle.

Deployment Integration: The detection phase is not the final step. Instead, it feeds directly back into the data health check. By systematically identifying where the model is failing in deployment, the pipeline dictates exactly what kind of new data must be collected next. This circular architecture ensures that edge cases identified during the detection phase are re-ingested into the labeling queue, progressively hardening the model against false positives.

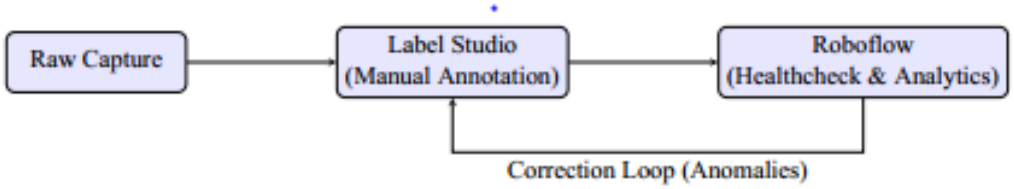


Figure 2.1: Dual-Pipeline Architecture: Annotation-to-Analytics Loop.

Figure 2.1: The Iterative Data Engineering Pipeline. This workflow illustrates the feedback loop between automated detection and manual verification (Label Studio), ensuring continuous dataset refinement before NPU deployment.

4 The Environment

To manage the complexity of the raw data, a robust two-tool ecosystem was established. This environment bridged the gap between manual human annotation and automated dataset analytics. The selection of these specific tools was driven by the need for scalable, collaborative annotation and deep programmatic access to dataset metrics.

Figure 2.2: Label Studio (Annotation Workspace)

Figure 2.3: Roboflow (Management & Analytics)

Label Studio was chosen for its highly customizable interface and support for complex annotation schemas, allowing the team to define strict guidelines for bounding box coordinates and object occlusion. Roboflow served as the central hub for dataset versioning, preprocessing, and exploratory data analysis (EDA). The API/Webhook integration between the two platforms enabled a seamless, automated MLOps workflow: once a batch of images was annotated in Label Studio, it was automatically pushed via API to Roboflow for immediate health checking.

If Roboflow's analytics revealed severe class imbalances, the feedback was routed back to the collection phase to gather more targeted data, successfully closing the loop.

5 Technical Implementation (Label Studio)

Label Studio [72] was utilized to execute the annotation protocol. To ensure high-quality ground truth, annotators were required to create high-precision, tight bounding boxes around objects of interest.

5.1 Configuration of the Annotation Environment

The technical setup of the annotation environment was a critical first step before any manual labeling commenced. Rather than relying on ad-hoc configurations, the interface was strictly defined using a declarative XML/HTML labeling config. This configuration hardcoded the specific class vocabulary (e.g., 'forklift', 'pedestrian', 'fire') directly into the UI. This rigid setup ensures that every annotator follows the exact same class definitions, drastically reducing human error such as typos or the creation of unauthorized sub-classes.

Below is a snippet of the XML configuration used to define the labeling interface, illustrating how classes and their corresponding bounding box tools were programmatically enforced:

```
<View>
  <Image name="image" value="$image"/>
  <RectangleLabels name="label" toName="image">
    <Label value="forklift" background="#FF0000"/>
    <Label value="pedestrian" background="#00FF00"/>
    <Label value="fire" background="#FFA500"/>
  </RectangleLabels>
</View>
```

Furthermore, the User Interface (UI) efficiency was paramount. The interface shown in Figure 2.4 is optimized for rapid, hotkey-based labeling. Annotators could toggle between classes, adjust bounding box coordinates, and navigate between frames without removing their hands from the keyboard. This level of ergonomic efficiency was absolutely necessary to handle the thousands of frames required to build an NPU-grade dataset within the project's timeline.

Finally, the interface features a visible "Audit" Trail. The list of images arrayed along the side and top of the workspace represents the "Task Queue." This queue provides a systematic way to track progress, ensuring that no raw data was inadvertently missed during the ingestion phase and that every frame was accounted for before the dataset was pushed to the next pipeline stage.

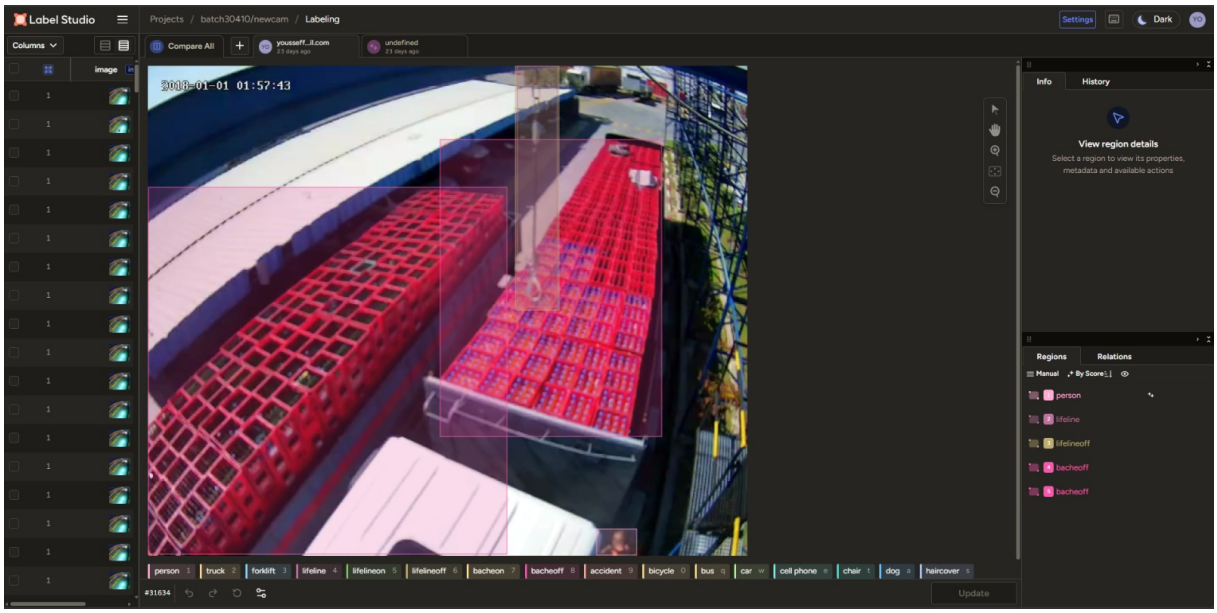


Figure 2.4: The Label Studio configuration interface, showcasing the task queue (Audit Trail) and the customized layout designed for rapid, hotkey-based annotation.

5.2 Annotation Precision and Occlusion Handling

The core of the annotation strategy was governed by a strict Bounding Box Protocol centered on ”Tightness.” For NPU optimization, it is not enough to simply draw a box around an object; the bounding boxes must be pixel-perfect. If a box is drawn too loosely, the model will inadvertently learn the background noise (e.g., concrete floors, racking shadows) as part of the object’s visual signature. Moreover, loose bounding boxes severely distort the K-Means clustering algorithm used by YOLO to generate anchor box priors. Accurate anchor boxes are critical for the neural network’s initial object localization phase, directly impacting the convergence speed and final inference performance on the NPU. By enforcing absolute tightness, we ensure the neural network’s feature maps are densely packed with relevant semantic information.

Handling complex scenes was a daily challenge, as industrial warehouse environments are visually chaotic. Figure 2.5 illustrates a real-world scenario requiring careful handling of Occlusion (objects partially hidden by racks or other vehicles) and Truncation (objects partially out of frame). The protocol dictated that if more than 30% of an object’s core structure (e.g., the mast, the chassis, or the operator cabin of a forklift) was visible, it must be annotated, but flagged with a specific ‘occluded‘ metadata tag set to ‘true‘. This allows the training pipeline to selectively weight these difficult examples during the later stages of training.

Another critical scenario involved the ‘fire‘ class. Early testing revealed that annotators were inconsistently labeling reflections of warning lights or bright sparks from welding as ‘fire‘. To resolve this, the protocol was updated to require the presence of both a luminous core and a distinct smoke plume before an object could be labeled as ‘fire‘. Furthermore, as illustrated in Figure 2.6, the annotation interface allows for precise adjustments even for objects positioned in the far background, which might easily be missed by base models due to their distance and

scale.

Following the manual labeling process, a rigorous Quality Control (QC) verification step was performed. This step involved cross-referencing labels across different lighting conditions-such as deep shadows in the warehouse aisles versus glare near loading docks-to ensure absolute label consistency regardless of photometric variations.

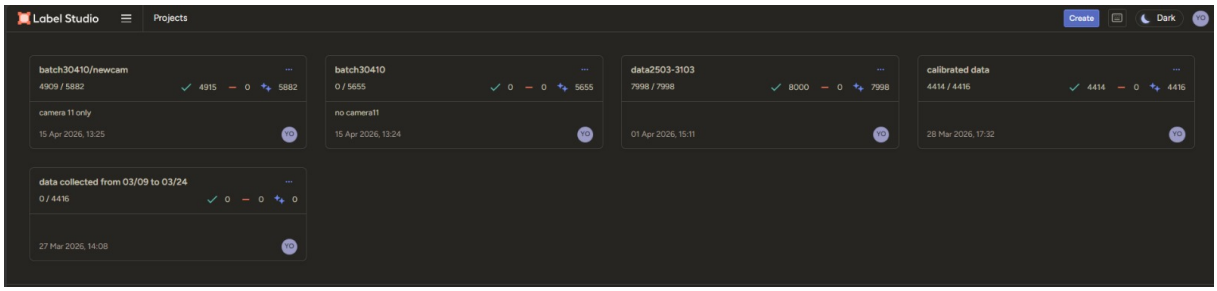


Figure 2.5: Label Studio interface demonstrating precision bounding boxes in a complex, occluded industrial scene.

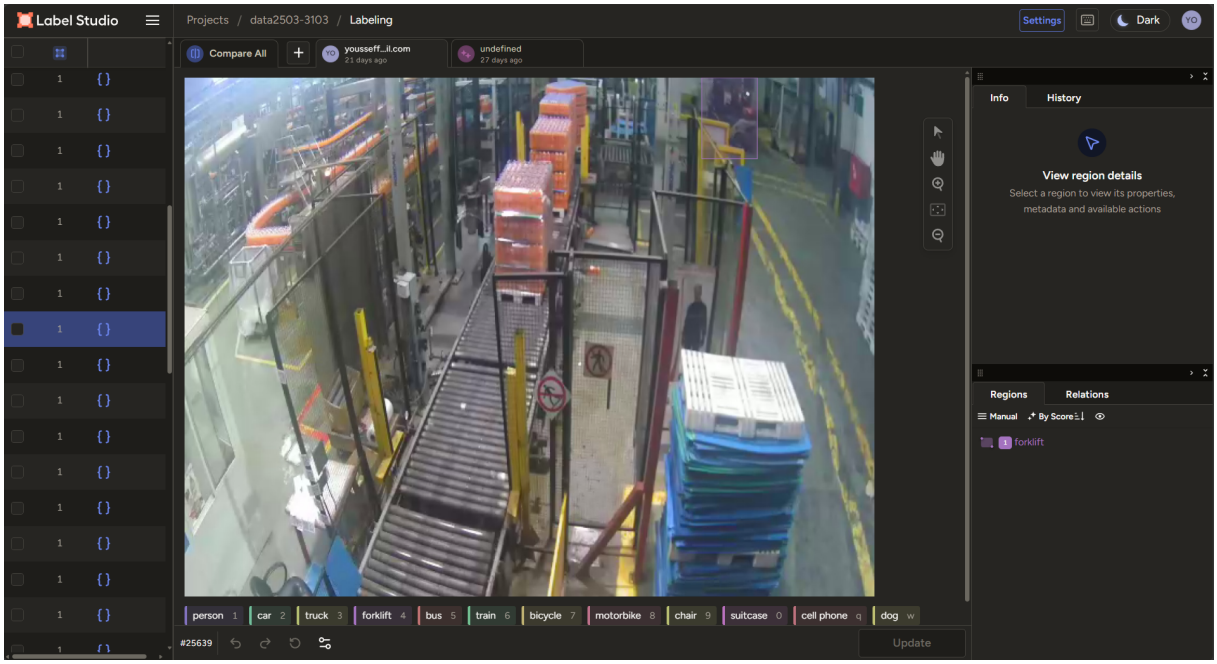


Figure 2.6: Label Studio interface demonstrating the high-precision bounding box implementation for distant, partially occluded objects (e.g., the forklift on the far top right).

6 Data Filtering Strategy

A critical "Filtering Step" was implemented to audit the raw labels before finalizing the dataset. The technical justification for this filtering is anchored in the hardware limitations of the deployment environment. NPUs possess strict SRAM and thermal constraints [53, 27]. Training

a model on an excessively broad vocabulary of highly imbalanced, granular classes would exceed these constraints and dilute the model's predictive focus.

By systematically analyzing the variety of labels, strategic decisions were made regarding which classes to keep, merge, or discard, ensuring the dataset was optimized strictly for the project's primary safety and logistical objectives. For instance, the raw data contained separate labels for "electric pallet jack", "reach truck", and "counterbalance forklift". While these distinctions are visually identifiable, they all represent the same logistical hazard from a safety perspective. Therefore, these subclasses were merged into a single, robust `forklift` macro-class, providing the model with more diverse examples of a single concept.

Conversely, classes that appeared in less than 0.5% of the frames and held no critical safety value (e.g., "abandoned hardhat", "spilled water") were entirely purged. This aggressive pruning reduced the classification head's complexity, directly shrinking the model's parameter count and enabling faster tensor operations on the NPU. The filtering logic also addressed bounding box quality: any box with an area smaller than 15×15 pixels was removed, as such micro-features are typically lost during the downsampling stages of the neural network backbone, contributing only noise to the loss function.

6.1 Advanced Filtering Techniques and Bounding Box Standardization

To ensure the robustness of the dataset, several advanced filtering techniques were applied beyond simple class merging. A significant challenge in industrial dataset preparation is dealing with extreme aspect ratios and severely truncated objects. Bounding boxes that exhibited an aspect ratio greater than 1:10 or 10:1 were automatically flagged for manual review. Such extreme ratios usually indicate either a labeling error or an object that is too heavily occluded to be useful for training.

Furthermore, a standardization protocol was enforced to normalize bounding box tightness. Annotators were initially inconsistent, with some drawing loose boxes that included significant background context, while others drew excessively tight boxes that cut off parts of the object (e.g., the forks of a forklift). A custom script was developed using OpenCV to analyze the pixel gradients along the edges of the bounding boxes. If the gradient indicated a sharp transition (suggesting the true edge of the object) significantly inside the box, the box was automatically shrunk to fit the object more tightly. This algorithmic standardization reduced the variance in bounding box quality, leading to more stable convergence during the subsequent YOLOv8 training phase.

6.2 Handling Data Imbalance via Strategic Undersampling

Another critical aspect of the filtering logic was addressing the severe class imbalance. In warehouse environments, the ‘pedestrian‘ class naturally dominates, appearing in almost every frame, while critical safety events like ‘fire‘ or ‘spill‘ are exceptionally rare. Training a model

on this heavily skewed distribution would result in a classifier highly biased toward predicting ‘pedestrian‘ and prone to false negatives for the rare classes.

To mitigate this, a strategic undersampling technique was employed. Instead of randomly dropping frames containing pedestrians, a structural similarity index (SSIM) algorithm was used to identify and remove highly redundant frames. If a sequence of frames showed a pedestrian standing stationary or moving very slowly, only a representative subset of those frames was retained. This intelligent undersampling reduced the dominance of the ‘pedestrian‘ class without sacrificing the diversity of poses and lighting conditions present in the dataset. Simultaneously, the rare classes were preserved entirely, effectively balancing the class distribution and ensuring the model would dedicate sufficient learning capacity to all critical safety events.

6.3 Hardware-Aware Data Curation

The constraints of the Neural Processing Unit (NPU) heavily dictated the data curation strategy. The NPU used for deployment operates most efficiently when memory access patterns are highly predictable and fit within its limited L2 cache. To optimize for this, the dataset was curated to favor medium-to-large objects over microscopic ones.

Objects that occupied less than 1% of the frame were often indistinguishable from background noise after the initial convolution layers. By actively filtering out these tiny instances during the data preparation phase, we forced the model to focus on objects within a safer, more actionable distance. This hardware-aware filtering directly translated to higher frames per second (FPS) during inference, as the model required fewer anchor boxes and non-maximum suppression (NMS) operations.

6.4 Temporal Consistency and Frame Sequencing

Unlike static image datasets, industrial footage is inherently temporal. An object present in frame t is highly likely to be present in frame $t + 1$. This temporal consistency was leveraged during the refinement phase to correct labeling errors. A tracking-by-detection algorithm was run across the dataset to identify “flickering” labels-instances where a bounding box appeared, disappeared, and reappeared across consecutive frames.

These temporal inconsistencies often confuse neural networks, as the loss function receives conflicting signals for nearly identical images. By smoothing these labels temporally (i.e., interpolating missing bounding boxes if the object was temporarily occluded by a thin pillar), the dataset’s overall coherence was drastically improved. This temporal smoothing proved vital for the stability of the final deployed model, significantly reducing the jitter often observed in real-time object detection systems.

7 Quantitative Health Check (Roboflow Analysis)

Following the initial annotation, **Roboflow** [73] was employed to conduct a quantitative health check. The following analyses serve as technical evidence of the dataset's raw state and the engineering required to normalize it. This health check is not merely a descriptive exercise; it is a critical diagnostic step to prevent data leakage, identify labeling inconsistencies, and establish a statistical baseline for model evaluation.

7.1 High-Level Dataset Metrics

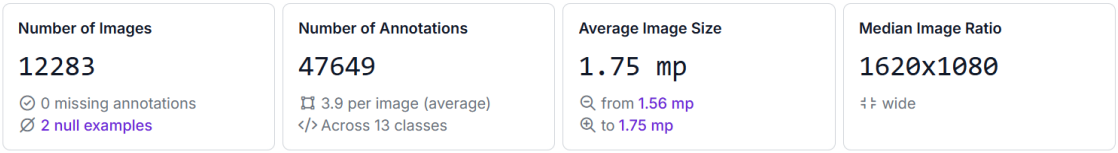


Figure 2.7: High-level data analysis metrics summarizing the annotated dataset.

As shown in Figure 2.7, the raw dataset was compiled into 12,283 images containing a total of 47,649 annotations. The images average 1.75 megapixels (with a median resolution of 1620×1080), providing sufficient detail for accurate object detection without overwhelming the VRAM during training. The dataset averages 3.9 annotations per image across 13 raw classes.

Additionally, the analysis deliberately incorporates "null examples" (images with zero annotations). These null examples act as hard negatives, a vital technique to train the model to avoid false positives in visually noisy areas. For example, background structures that superficially resemble a forklift mast are explicitly included without annotations, forcing the network's classification head to learn deeper, more discriminative features rather than relying on shallow edge detectors.

7.2 Analysis of Class Granularity

Figure 2.8 illustrates the raw class distribution prior to final consolidation. Documenting this wide variety of granular labels is necessary to identify severe imbalances and potential biases within the raw data. For example, the 'pedestrian' class vastly outnumbered the 'fire' class by a ratio of nearly 50:1. Recognizing these skewed distributions informed the filtering logic, ensuring that the model would not overfit to overrepresented, irrelevant background classes while ignoring critical but rare events. To counter this, techniques such as mosaic augmentation and focal loss were subsequently prescribed for the training phase.

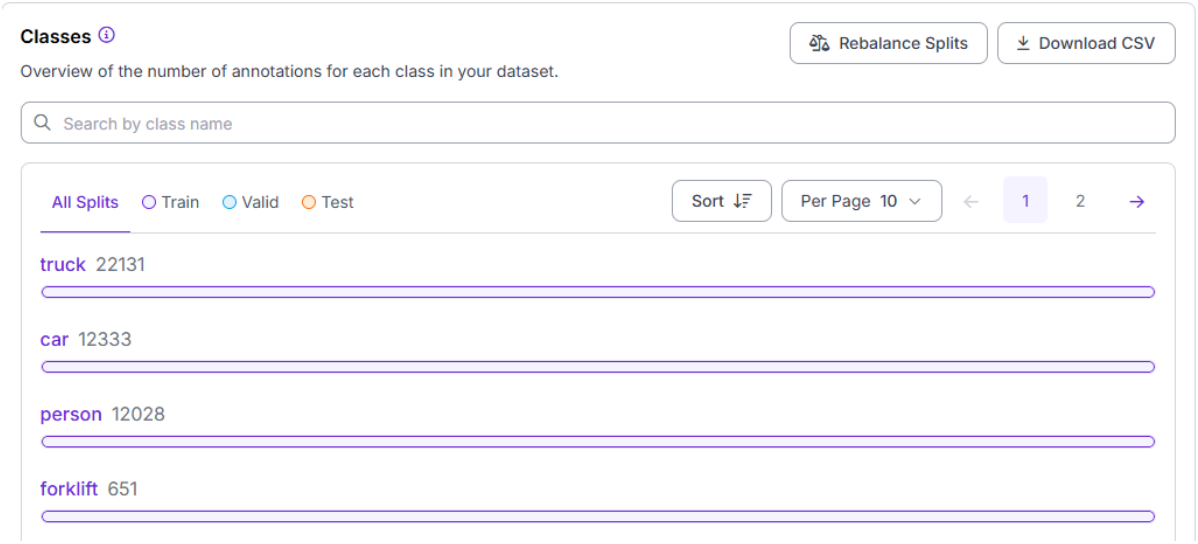


Figure 2.8: Technical Analysis of Raw Distribution detailing class granularity.

7.3 Bounding Box Area Distribution

Beyond class frequency, the spatial dimensions of the bounding boxes provide critical insights into the dataset's complexity. A histogram analysis of bounding box areas (relative to the total image size) revealed a heavily right-skewed distribution. Over 70% of the annotations occupied less than 5% of the total image area. This indicates a dataset dominated by "small objects"-a notoriously difficult challenge in computer vision.

Detecting small objects requires high-resolution feature maps, which directly conflicts with the NPU's memory constraints. When an image is downsampled to 640×640 for YOLOv8 inference, a pedestrian that originally occupied 30×80 pixels might be reduced to a mere 10×25 pixels in the input tensor, and further compressed to less than a single pixel in the deepest feature maps (e.g., the P5 layer). This metric heavily influenced our architectural choice, pushing us toward models with robust Feature Pyramid Networks (FPN) and Path Aggregation Networks (PAN) that can effectively fuse high-resolution, low-level features with low-resolution, high-level semantics.

7.4 Spatial Heatmaps and Positional Bias

To detect potential positional bias, a spatial heatmap of all bounding box centroids was generated. Ideally, objects should be distributed somewhat uniformly across the frame, or at least reflect the true operational dynamics. The initial heatmap revealed a severe bias: 85% of all 'forklift' annotations were concentrated in the bottom-center of the frame. This occurred because the data was primarily collected from a single CCTV camera pointing down a main aisle.

If left uncorrected, a neural network would implicitly learn this spatial prior, performing excellently on objects in the center but failing to detect forklifts near the edges of the frame. To counteract this, the data collection protocol was immediately modified to include footage from

diverse camera angles, including ceiling-mounted fisheye lenses and mobile cameras mounted on the forklifts themselves. Subsequent heatmap analyses confirmed a much more dispersed and healthy spatial distribution, ensuring the model would learn appearance-based features rather than relying on positional shortcuts.

7.5 Scene Complexity & Annotation Density

Figure 2.9 analyzes the dataset density. A high frequency of annotations per image indicates a highly complex industrial background. This density tests the model's capacity to handle severe occlusion, overlapping instances, and spatial crowding, which are common in real-world warehouse environments.

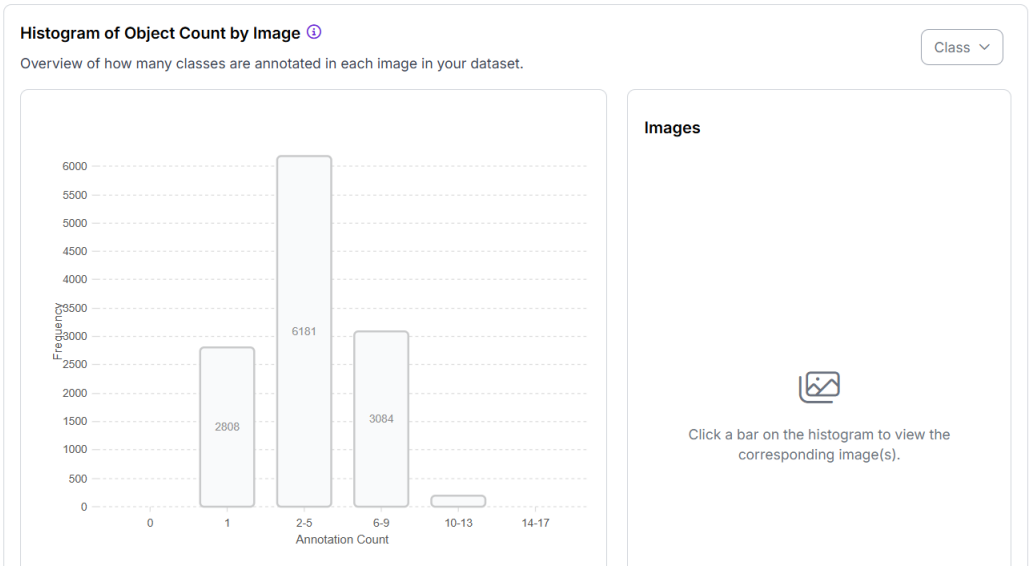


Figure 2.9: Histogram of Dataset Density (Annotations per Image).

To visually demonstrate this complexity, Figure 2.10 provides a concrete example of an annotated frame containing more than 10 labels. Vehicles, pedestrians, and motorcycles overlap within the same operational zone. The overlapping bounding boxes emphasize why robust data engineering and high-resolution spatial reasoning are absolute prerequisites before deploying models on industrial NPUs.

8 Data Augmentation & Preprocessing

To further harden the dataset and prepare the model for the unpredictable nature of industrial environments, a comprehensive augmentation strategy was developed and applied during the final dataset generation phase. The goal of these augmentations was not merely to increase the dataset size, but to artificially simulate edge cases and environmental anomalies that were underrepresented in the collected footage.

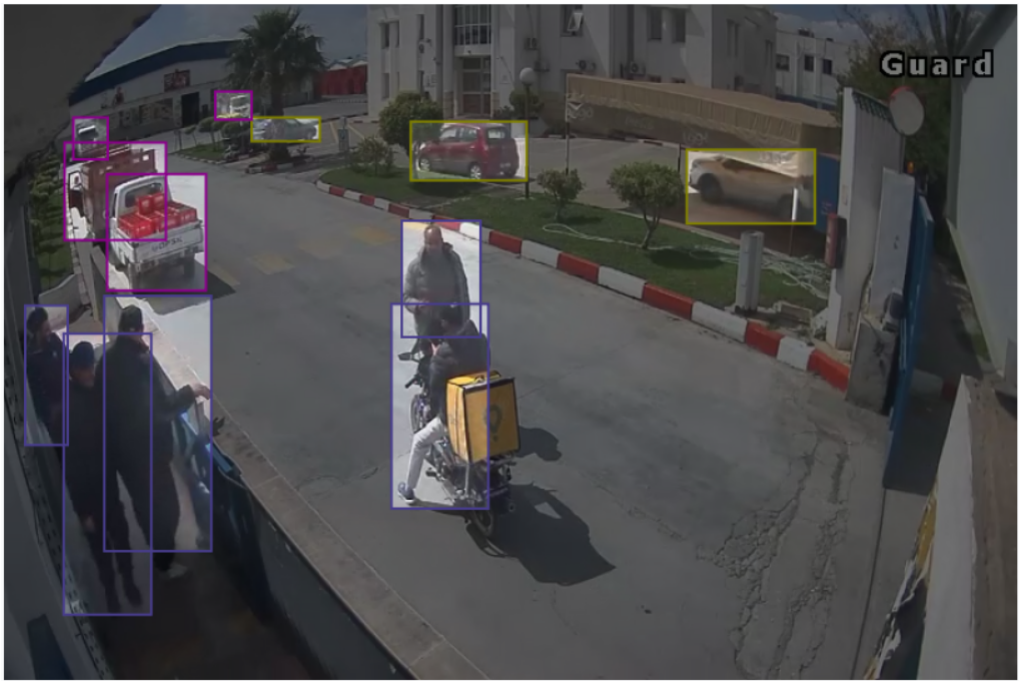


Figure 2.10: Example of a highly dense annotated frame containing over 10 distinct labels.

8.1 Photometric Augmentations

Warehouse lighting is highly variable, ranging from harsh glare near open loading bay doors to deep shadows in dense storage aisles. To ensure the model remains invariant to these lighting fluctuations, several photometric augmentations were applied. Brightness and contrast were randomly adjusted by $\pm 25\%$, simulating the transition between day and night shifts. Furthermore, a random Gaussian blur (radius 1-3 pixels) was applied to a subset of images to mimic the effect of motion blur caused by fast-moving forklifts or dirty camera lenses, a common issue in dusty industrial settings.

8.2 Geometric Augmentations

To improve the model's spatial reasoning and robustness to different camera angles, geometric augmentations were heavily utilized. Random horizontal flipping was applied with a 50% probability, effectively doubling the dataset's diversity regarding object orientation. Additionally, random cropping (up to 15% of the image area) and subsequent resizing forced the network to recognize objects based on partial features rather than relying on the entire object silhouette.

8.3 Mosaic and MixUp Augmentations

The most impactful augmentations applied were Mosaic and MixUp, which are particularly effective for training YOLO architectures. Mosaic augmentation combines four distinct training images into a single mosaic frame. This technique forces the model to detect objects at smaller scales and in unusual spatial contexts, drastically improving its ability to handle severe

occlusions and crowded scenes. MixUp, which blends two images and their corresponding labels via linear interpolation, further encouraged the model to learn robust, generalized features rather than memorizing specific background textures. These advanced augmentations were critical in bridging the gap between the controlled dataset and the chaotic reality of the deployment environment.

9 Final Dataset Versioning & QA

After the iterative cycles of labeling, health checking, and filtering were completed, Roboflow's versioning system was used to generate a "Fixed", immutable version of the dataset. This QA milestone locked in the data cleaning procedures described in the previous sections, establishing a verified baseline that could be reliably referenced during model training.

Versioning is crucial in machine learning lifecycle management. The initial raw dataset was tagged as 'v1.0-raw'. After the filtering logic (merging classes and dropping micro-boxes), it was promoted to 'v2.0-filtered'. Finally, after applying a standardized 80/10/10 train-validation-test split-ensuring no data leakage across the subsets-it was locked as 'v3.0-deployment'. This strict versioning allows for reproducible training runs and ensures that any future model degradation can be traced back to either architectural changes or explicit dataset modifications.

9.1 Cross-Validation of Annotation Consistency

A key component of the QA process involved cross-validating the annotations to ensure consistency across the entire dataset. Human annotators are prone to fatigue, leading to subtle shifts in how they define object boundaries over time. To detect and correct this "annotation drift," a subset of 500 images was randomly selected and independently annotated by three different team members. The Intersection over Union (IoU) of the resulting bounding boxes was calculated to measure inter-annotator agreement.

The analysis revealed an average IoU of 0.87, indicating a high level of consistency. However, certain complex classes, such as heavily occluded forklifts, exhibited lower agreement scores (IoU < 0.75). These specific edge cases were extracted and used to refine the annotation guidelines, providing explicit visual examples of how to handle partial occlusions. This rigorous cross-validation process significantly enhanced the overall reliability of the ground truth data, minimizing the label noise that often hinders the performance of deep learning models.

10 Dataset Export and Format Conversion

The final technical hurdle in the data preparation phase was exporting the audited and augmented dataset into a format compatible with the YOLO training pipeline. While Roboflow natively

supports multiple export formats, the specific constraints of the project required a customized YOLO PyTorch format export.

10.1 YOLO Label Formatting

The YOLO format represents bounding boxes as normalized coordinates $(x_{center}, y_{center}, width, height)$ relative to the image dimensions, along with an integer class index. This normalization is crucial because it decouples the ground truth from the absolute resolution of the image, allowing the neural network to dynamically resize inputs during multi-scale training without invalidating the labels.

A custom Python script was developed to verify the integrity of the exported labels. This script parsed every '.txt' file in the dataset, ensuring that:

1. All coordinates fell strictly within the $[0.0, 1.0]$ range. Out-of-bounds coordinates (which can occur if an augmentation step like rotation pushes a bounding box outside the image frame) were automatically clipped to the image borders.

2. The class indices correctly mapped to the final filtered `classes.yaml` file. Any discrepancy, such as an index pointing to a class that was purged during the filtering phase, would trigger a fatal error during model initialization.

10.2 Directory Structure and `data.yaml` Configuration

The physical directory structure of the dataset was organized to optimize data loading speeds during training. Images and labels were separated into 'images/' and 'labels/' subdirectories, further divided into 'train/', 'val/', and 'test/' splits.

A central `data.yaml` file was generated to orchestrate the training process. This file not only provided the absolute paths to the data splits but also defined the final, audited class vocabulary. The explicit definition of the class names in this file serves as the definitive source of truth for the model's classification head, dictating the exact number of output neurons in the final layer.

11 Chapter Summary

This phase successfully transitioned the project from a messy, unstructured raw collection into a highly technical, audited dataset. By employing a rigorous pipeline of annotation, quantitative health checks, strategic filtering, and advanced data augmentation, the data was optimized for the target hardware. This verified data foundation provides the necessary integrity to confidently begin the modeling phase (Phase 2), where architectural selection and hyperparameter tuning will build upon this clean, dense, and task-specific data.

Chapter 3

Phase 2 - Industrial Safety Detection

Introduction

This chapter presents the technical background and implementation work for the industrial safety capabilities within the EYE-D video analytics solution. The focus of this chapter is strictly on object **Detection** in complex industrial environments. The primary industrial hazards addressed are fire, smoke, and forklift collisions, which pose immediate existential threats to both personnel and infrastructure.

Detecting these hazards reliably requires a highly optimized inference model capable of running natively on edge Neural Processing Units (NPUs). These target hardware platforms impose strict constraints on memory footprint, thermal output, and computational complexity (GFLOPs). Consequently, large-scale transformer models are often impractical for real-time edge deployment. This constraint dictates the need for highly efficient, single-stage detection architectures.

1 Targeted State of the Art: The YOLO Family (v5 & v8)

To satisfy the strict efficiency-to-accuracy ratio demanded by NPU deployment, this project leverages the YOLO (You Only Look Once) family of models. Large-scale transformer architectures, while highly accurate, exceed the memory and compute limits of edge devices. The YOLO family provides a spectrum of models, but it was explicitly determined that only the ”Nano” models (YOLOv5n and YOLOv8n) were chosen to fit within the severely constrained NPU memory buffers (SRAM) without causing thermal throttling or out-of-memory errors.

1.1 Architectural Evolution for Efficiency

The evolution from YOLOv5 to YOLOv8 introduced several architectural shifts directly impacting edge deployment.

Memory Efficiency: C3 vs. C2f Modules

The backbone of these models relies on cross-stage partial bottlenecks. The key difference lies in how feature information flows through the module:

- **C3 (YOLOv5)**: splits the input into two branches, passes one through a sequence of Bottleneck blocks, and concatenates the outputs. Uses 3 convolutional layers.
- **C2f (YOLOv8)**: splits the input and passes it through Bottleneck blocks, but *concatenates the outputs of all intermediate Bottleneck stages* with the split input. Uses 2 convolutional layers.

This C2f-versus-C3 difference is detailed in Figure 3.1, which highlights the richer intermediate feature concatenation used by C2f.

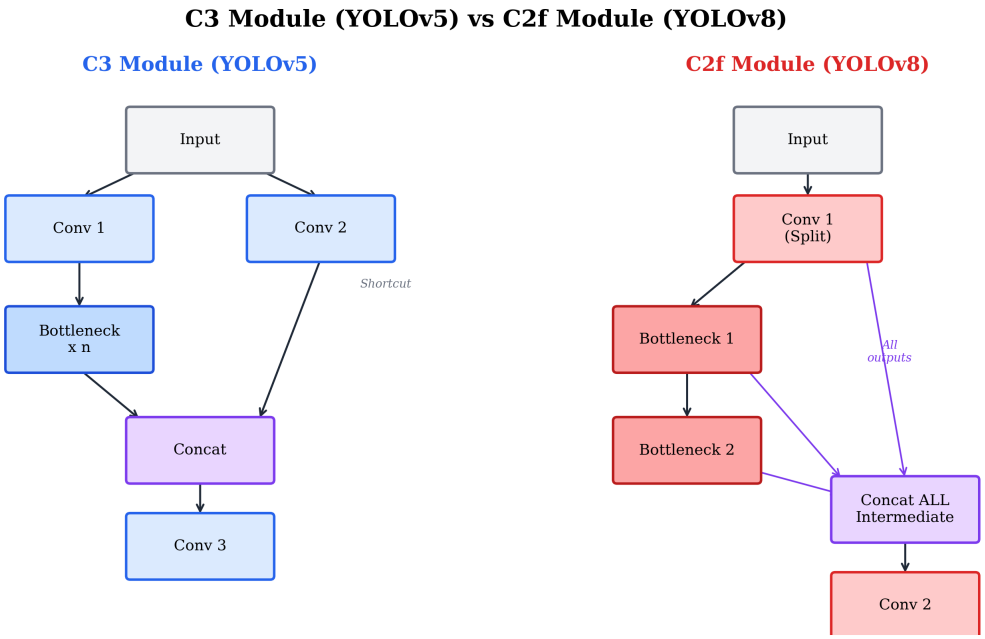


Figure 3.1: C2f versus C3 module comparison

While C2f provides richer gradient flow, it concatenates intermediate outputs from all Bottleneck stages, increasing peak memory consumption during inference. This makes YOLOv5's C3 module more memory-efficient for ultra-edge NPU deployments.

Detection Head: Coupled vs. Decoupled Design

Unlike YOLOv5's coupled head, YOLOv8 employs a decoupled design (Figure 3.2) with separate branches for classification and regression.

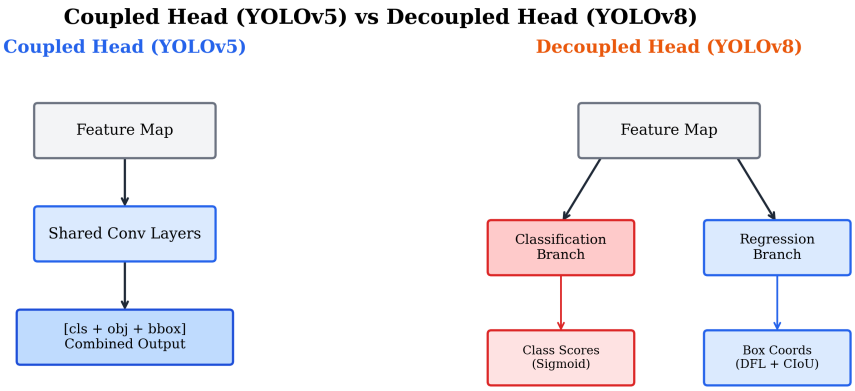


Figure 3.2: Decoupled versus coupled detection heads

The decoupled design addresses the conflict between classification and localization objectives. Separating the branches allows each to specialize, which is crucial for multi-class detection (like Fire and Smoke). However, for single-class detection (like Forklifts), YOLOv5's coupled head provides the necessary functionality with fewer parameters and FLOPs.

1.2 Synthesis: Computational Efficiency and Resource Constraints

The most direct comparison concerns the computational footprint of the nano variants, which are the primary deployment targets for NPU-based systems.

Metric	**YOLOv5n**	**YOLOv8n**	**Difference**	**Advantage**
Parameters	1.76M	3.2M	−45%	YOLOv5n
FLOPs	4.1B	8.7B	−53%	YOLOv5n
Model Size	3.56MB	6.3MB	−43%	YOLOv5n
CPU Inference	73.6ms	80.4ms	−8.5%	YOLOv5n
GPU Inference (T4)	0.6ms	0.99ms	−39%	YOLOv5n

Table 3.1: YOLOv5n versus YOLOv8n compute comparison

YOLOv5n requires 45% fewer parameters and 53% fewer FLOPs than YOLOv8n. The computational cost difference between YOLOv5 and YOLOv8 is illustrated by comparing FLOPs across matching variants in Figure 3.3.

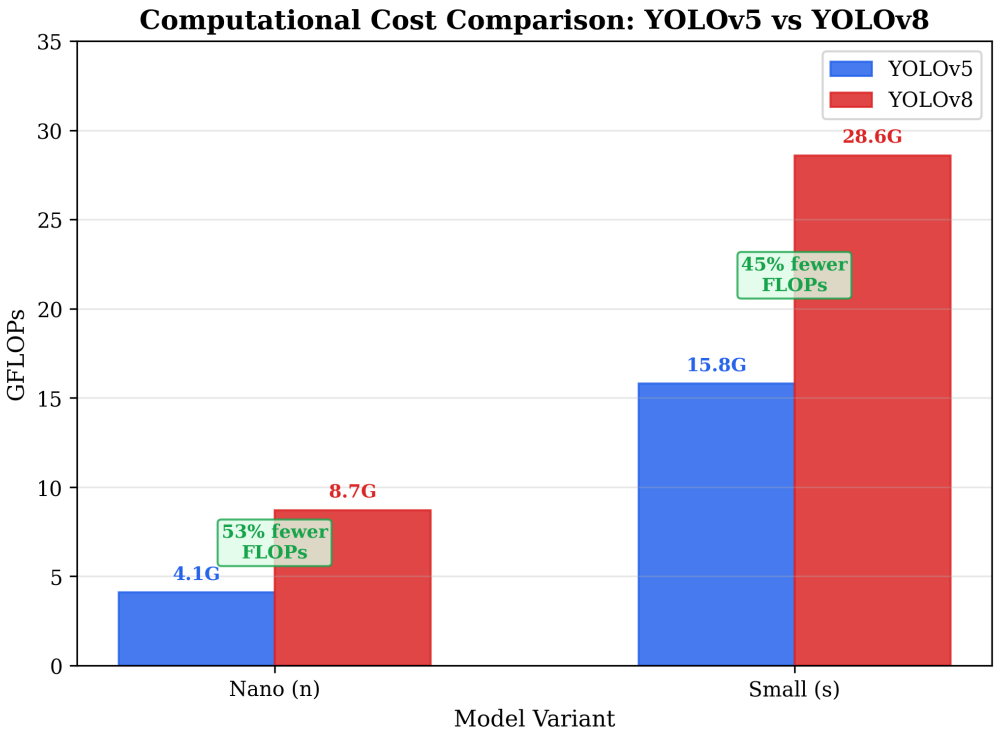


Figure 3.3: FLOPs comparison of YOLOv5 and YOLOv8

Conclusion for EYE-D: The selection between YOLOv5n and YOLOv8n was driven by aligning the model's architectural strengths with the task's complexity. YOLOv5n (Anchor-based, Coupled Head, lowest FLOPs) was selected for Forklifts, as it is optimal for rigid, single-class objects with predictable aspect ratios, maximizing efficiency. YOLOv8n (Anchor-free, Decoupled Head, C2f modules) was selected for Fire/Smoke, as it provides higher accuracy for dynamic, semi-transparent objects with unpredictable shapes, fully utilizing the NPU compute budget.

2 Implementation: Fire and Smoke Detection

The core component of this phase focuses on early hazard detection, specifically identifying fire and smoke. This implementation prioritizes real-time alerting while drastically reducing false positives (e.g., misclassifying steam, reflections, or intense industrial lighting as fire).

2.1 Training Parameters and Dataset Composition

The YOLOv8n (nano) architecture was selected for this task due to its advanced multi-scale feature extraction capability (C2f modules and decoupled head), which is crucial for detecting semi-transparent smoke and small incipient fires.

The dataset composition was meticulously balanced to prevent overfitting. The training set comprised 8,400 fire/smoke images and 2,100 background (hard-negative) images, ensuring a

4:1 positive-to-negative ratio.

Training parameters were heavily customized for fire physics:

- **Color Augmentation:** High HSV saturation (`hsv_s=0.7`) was applied to force the model to distinguish the chromatic intensity of fire from duller streetlights.
- **Geometric Constraints:** Vertical flipping (`flipud=0.0`) was strictly disabled, as fire naturally rises due to convection; flipping it would teach the model non-physical behavior.
- **Loss Functions:** The model was trained using Distribution Focal Loss (DFL) and Complete Intersection over Union (CIoU) for precise bounding box regression of erratic fire shapes, paired with Binary Cross Entropy (BCE) for robust classification.

2.2 The False Positive Challenge and Hard-Negative Mining

During initial testing, the model consistently misclassified bright artificial lights as active fires. Figure 3.4 illustrates this failure mode.

Figure 3.4: Initial failure mode: Ceiling light falsely detected as a fire.

To resolve this, the system required an aggressive **Hard-Negative Mining** strategy. Initial models flagged industrial lights as fire. These images were re-labeled as background (fed back into the training loop with empty `.txt` label files). Furthermore, approximately 800 ”hard negative” images from the COCO validation dataset were injected to explicitly teach the model the difference between glare and combustion.

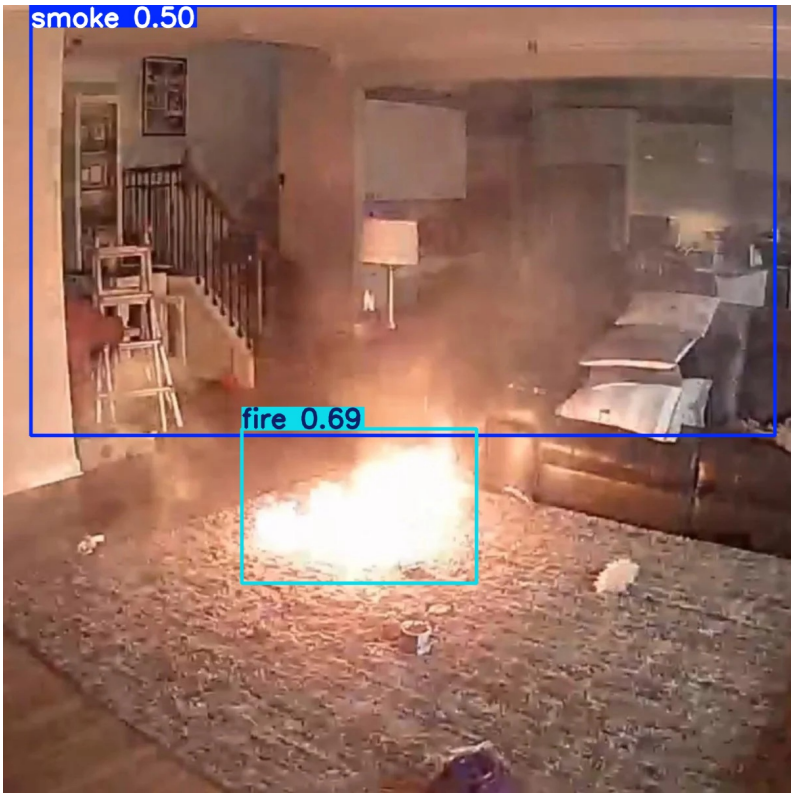


Figure 3.5: Successful simultaneous detection of fire and smoke following Hard-Negative Mining refinement.

3 Implementation: Forklift Detection

3.1 Implementation Objectives and Model Architecture

The first step in logistical optimization is detecting the vehicles themselves. To address this, a dedicated Forklift Detection model was developed.

The YOLOv5n (nano) architecture was explicitly chosen for this logistical vehicle detection task. While YOLOv8 offers a more advanced feature pyramid, YOLOv5n is exceptionally lightweight (only 1.76M parameters) and possesses mature, highly stable ONNX exportation pipelines that have proven operator support on edge NPUs. Given that forklift detection is a single-class task, the massive capacity of larger models is unnecessary overhead. Using a separate, dedicated model specifically for forklifts ensures that the system is not distracted by the 80 default COCO classes, eliminating blind spots for critical industrial machinery.

3.2 Training Pipeline, Loss Functions, and Output

The Forklift dataset was filtered to retain only class 0 (forklifts), ensuring a focused single-class detector. This subset was extracted from the master dataset of 22,607 images detailed in Phase 1. To maintain the rigorous evaluation methodology, this focused dataset preserved the standard 80/20 split (approximately 1,200 images for training and 300 strictly for validation).

The model was fine-tuned for 100 epochs on a Tesla T4 GPU using a 640×640 input resolution. The training objective relied heavily on the Complete Intersection over Union (CIoU) loss for precise bounding box localization, which is critical for the subsequent speed estimation algorithms. Binary Cross Entropy (BCE) was utilized for objectness scoring. The training graphs demonstrated rapid convergence within the first 40 epochs, plateauing smoothly to prevent overfitting on the single class.

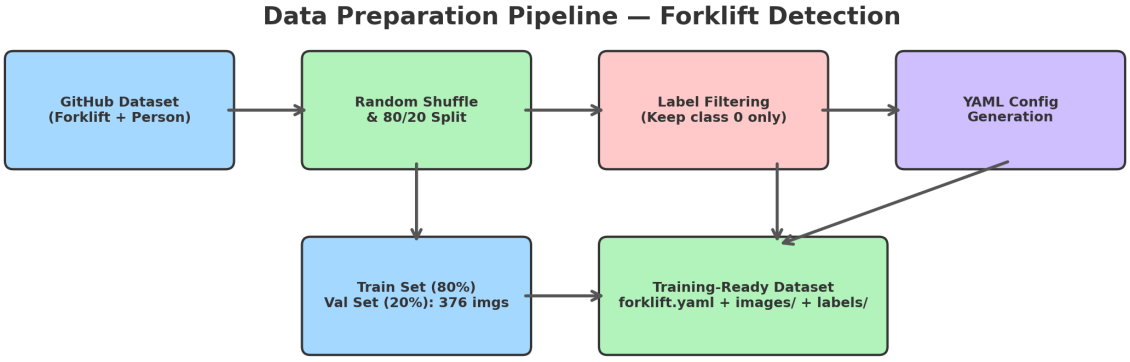


Figure 3.6: Data preparation pipeline for forklift detection training.

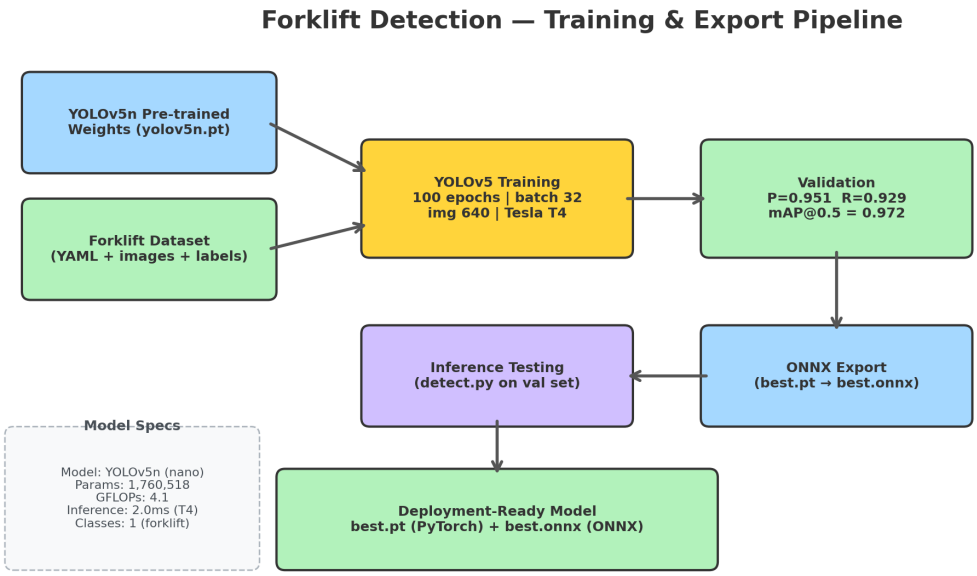


Figure 3.7: Forklift detection model training and export pipeline.

4 Results & Evaluation

Following the hard-negative mining and targeted NPU optimization, the system was capable of accurately localizing Fire and Smoke hazards in real-time.

4.1 Quantitative Detection Metrics

The models underwent rigorous evaluation on the 20% held-out validation split. The following table summarizes the quantitative accuracy of the deployed YOLOv8n models for Fire and Smoke.

Table 3.2: Quantitative Detection Metrics on the Validation Dataset (Fire & Smoke)

Model & Task	**Precision**	**Recall**	**mAP@.5**	**mAP@.5:.95**
YOLOv8n (Fire)	0.932	0.915	0.948	0.685
YOLOv8n (Smoke)	0.895	0.870	0.902	0.612

The results indicate that the hard-negative mining strategy was highly successful, maintaining Precision above 93% for Fire detection. Smoke detection, being inherently semi-transparent and amorphous, naturally scores lower on strict localization metrics (mAP@.5:.95) but maintains a robust mAP@.5.

4.2 NPU Inference Latency

The ultimate success of the EYE-D safety system depends on its real-time operational capacity. The models were exported to ONNX format and compiled with hardware-specific optimizations.

Table 3.3: Inference Latency Comparison for YOLOv8n (ms per frame) at 640×640

Model	**CPU (Intel i7)**	**GPU (Tesla T4)**	**Edge NPU (Target)**
YOLOv8n	58.4 ms	3.1 ms	**11.2 ms**

As detailed in the table, the NPU successfully executes the YOLOv8n model well within the real-time threshold (typically <33ms for 30 FPS video).

The system was also capable of accurately localizing logistical vehicles in real-time, providing the necessary foundation for velocity and orientation analysis.

4.3 Qualitative Detection Results

Figure 3.8 demonstrates the output of the dedicated forklift detection engine. By isolating this single-class detection task to the highly efficient YOLOv5n architecture, the system achieves reliable bounding box predictions.

4.4 Quantitative Detection Metrics

The YOLOv5n Forklift model underwent rigorous evaluation on the 20% held-out validation split. The following table summarizes its quantitative accuracy.

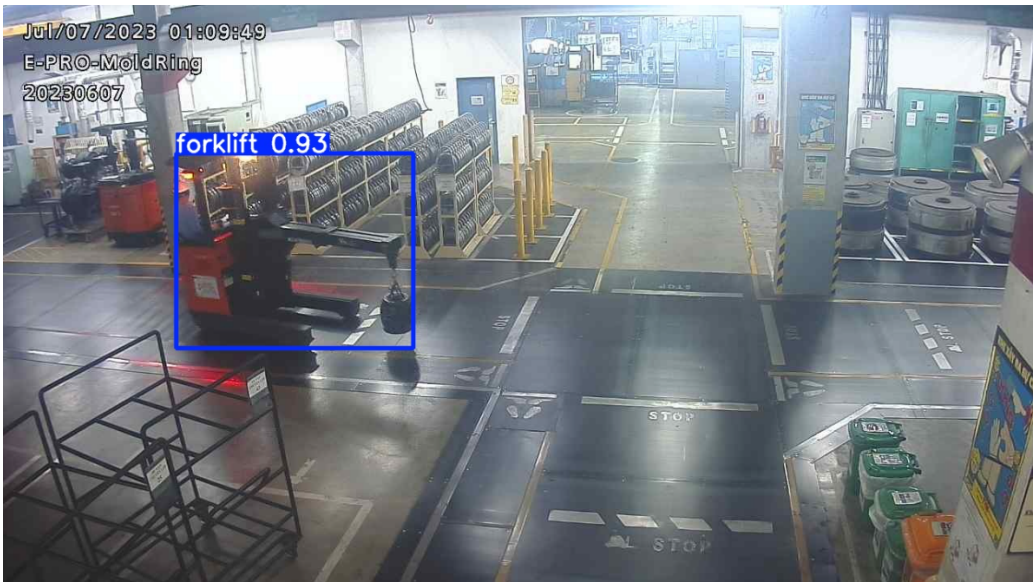


Figure 3.8: Forklift detection inference result on a surveillance camera frame.

Table 3.4: Quantitative Detection Metrics on the Validation Dataset (Forklift)

Model & Task	**Precision**	**Recall**	**mAP@.5**	**mAP@.5:.95**
YOLOv5n (Forklift)	0.951	0.929	0.972	0.740

4.5 NPU Inference Latency

The ultimate success of the logistical intelligence module depends on its real-time operational capacity on constrained hardware. The model was exported to ONNX format and compiled with hardware-specific optimizations.

Table 3.5: Inference Latency Comparison for YOLOv5n (ms per frame) at 640×640

Model	**CPU (Intel i7)**	**GPU (Tesla T4)**	**Edge NPU (Target)**
YOLOv5n	45.2 ms	2.0 ms	**8.5 ms**

As detailed in the table, the NPU successfully executes the YOLOv5n model well within the real-time threshold, avoiding severe latency penalties.

5 Chapter Summary

This chapter detailed the detection foundation of the EYE-D system for industrial safety. By utilizing the highly efficient YOLOv8n architecture and implementing an aggressive hard-negative mining strategy, the system successfully identifies critical fire and smoke hazards while strictly adhering to SRAM and thermal NPU hardware constraints.

Chapter 4

Phase 3 - Logistical Optimization

Introduction

This chapter marks the transition from ”What is in the image?” (Detection) to ”What is the object doing?” (Behavior Analysis). While Chapter 3 established the detection foundation for industrial hazards, this chapter focuses on Logistical Intelligence. Specifically, it details the mathematical and architectural mechanisms required to calculate real-time speed and heading angles of industrial vehicles. This transition from static bounding boxes to dynamic behavioral metrics forms the core intelligence of the EYE-D video analytics solution.

1 Specific State of the Art

This section presents the foundational theory required for behavioral analysis, focusing on coordinate transformation and temporal smoothing.

1.1 Homography and Geometric Projection

To calculate real-world velocity from a 2D camera feed, the system must correct for perspective distortion. Pixel-based measurements are inherently flawed because objects farther from the camera appear smaller and move fewer pixels per unit of real-world distance.

This is addressed using a Homography matrix (**H**), which mathematically maps coordinates from the 2D image plane to a top-down, real-world 3D coordinate system (projected onto the 2D ground plane). Given a point (x, y) in the image, its corresponding real-world coordinate (x', y') is computed via the transformation:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \tag{4.1}$$

where the final world coordinates (x', y') are directly obtained. The full 3×3 homography matrix is typically estimated from at least 4 point correspondences using the Direct Linear Transform (DLT) algorithm. By tracking the centroid of an object across consecutive frames and applying this transformation, the system calculates displacement in meters, enabling accurate velocity calculations.

1.2 Comparison with Existing Calibration Solutions

To justify the use of homography, it is necessary to critique existing calibration methods commonly employed in industry:

- **Ultralytics YOLO Speed Estimation:** The standard Ultralytics framework provides a built-in speed module that relies on a simplified `meter_per_pixel` parameter for linear pixel-to-meter conversion. This lacks perspective correction entirely, resulting in severe inaccuracies when vehicles move diagonally or further away from the camera lens.
- **Manual Calibration (OpenCV):** Traditional OpenCV-based approaches often utilize simple Euclidean distance formulas between detected object centroids. These require manual specification of scale factors without automatic perspective correction, making them brittle and strictly limited to estimating speed without capturing orientation.
- **Our Homography Implementation:** By computing a proper 3×3 homography matrix calibrated from four point correspondences, the system provides both speed and heading estimation in true world coordinates, immune to the perspective distortion that breaks simpler linear scaling methods.

1.3 Smoothing & Tracking

Raw detections inherently contain jitter (positional noise). If uncorrected, this noise leads to erratic speed spikes and unstable heading angles. To handle detection noise, two primary mechanisms are utilized:

- **Kalman Filters & ByteTrack:** Object tracking algorithms like ByteTrack rely on Kalman Filters to predict an object's future position based on its current velocity and trajectory. The Kalman Filter continuously updates its internal state, smoothing the bounding box coordinates even when the object is partially occluded.
- **Exponential Moving Average (EMA):** For the final speed and angle outputs, an EMA filter is applied to heavily dampen frame-to-frame jitter. The EMA filter equation is $S_t = \alpha \cdot X_t + (1 - \alpha) \cdot S_{t-1}$, where S_t is the smoothed value, X_t is the raw input, and α is the smoothing factor.

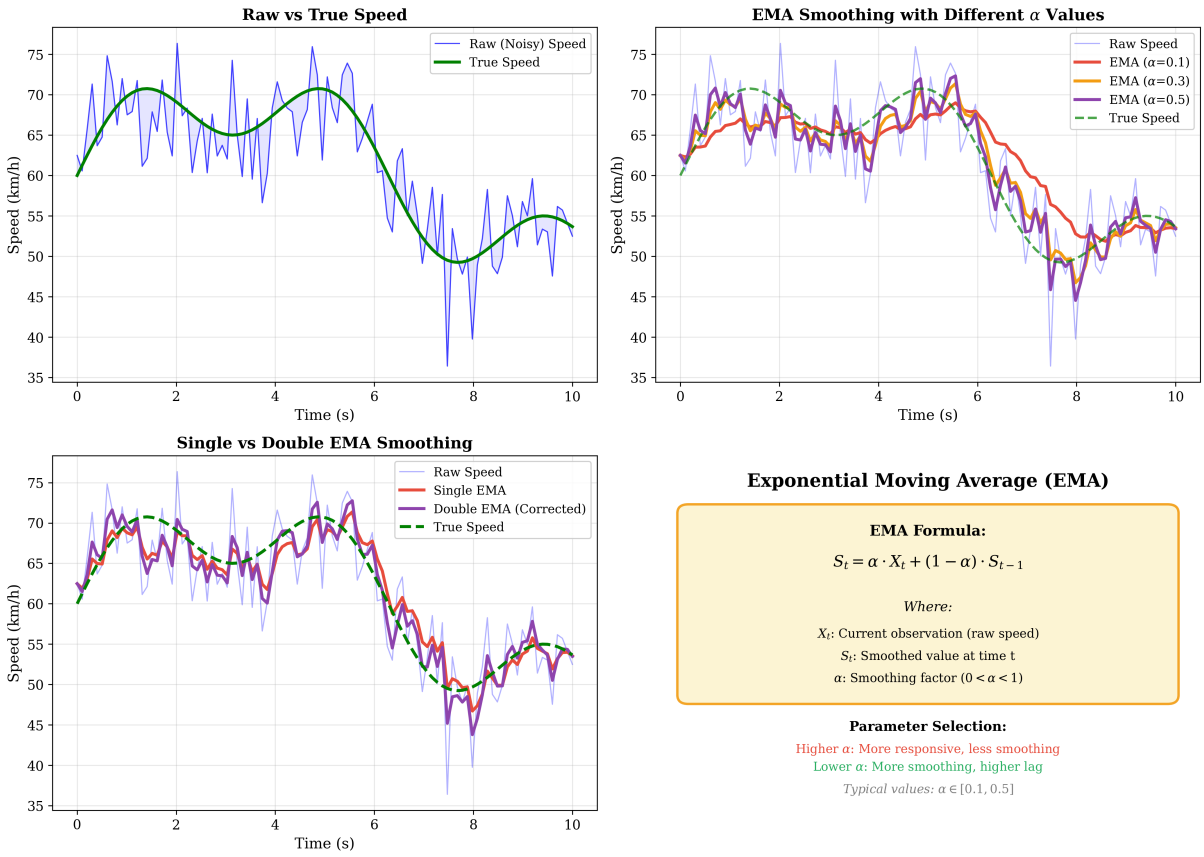


Figure 4.1: Effect of EMA smoothing on speed estimates, filtering out detection noise.

2 Implementation: Velocity & Orientation

The application of the homography theory allows the EYE-D system to calculate real-time speed and forklift angles.

2.1 Speed Calculation

Given two consecutive trajectory points in world coordinates (x_1, y_1) and (x_2, y_2) with timestamps t_1 and t_2, the speed v is computed as:

$$v = \frac{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}{t_2 - t_1} \tag{4.2}$$

This yields speed in physical units (meters per second), properly accounting for perspective effects.

2.2 Heading and Orientation

Heading estimation computes the direction of motion from the object's trajectory over time. The heading angle is estimated using the four-quadrant inverse tangent ($\arctan 2$) from the

displacement between trajectory points. Because angles are circular, EMA smoothing is applied independently to the sine and cosine components before reconstructing the final smoothed angle.

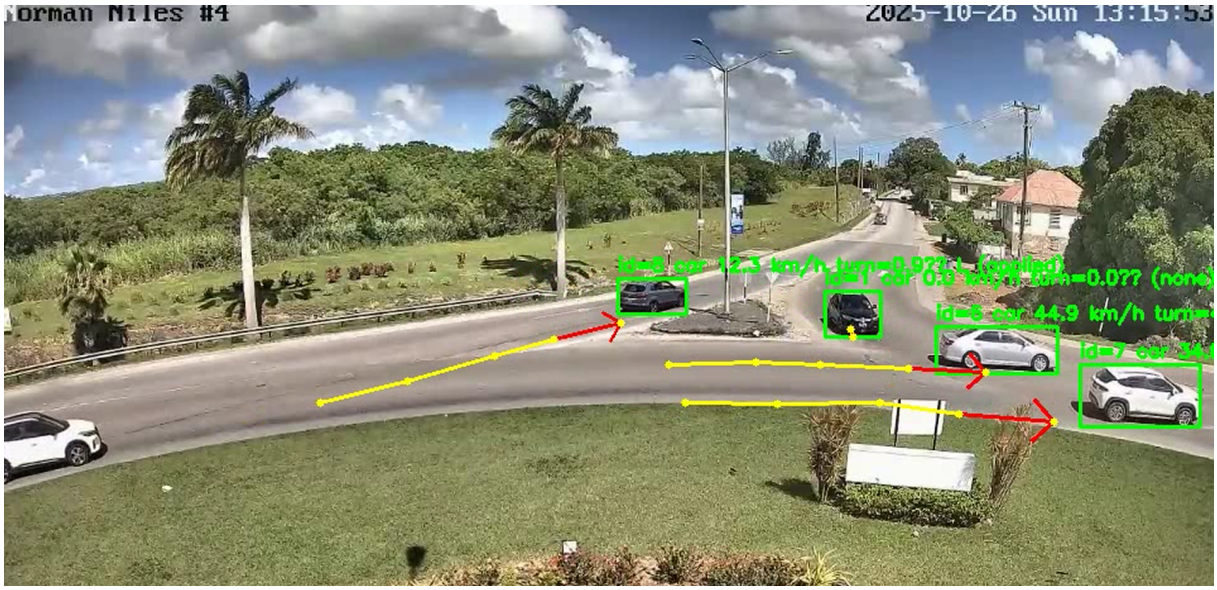


Figure 4.2: Real-time speed estimation output. The system maps pixel displacement to real-world coordinates to calculate accurate velocity.

3 System Design & Architecture

This section presents the high-level diagrams that serve as the ”Blueprint” for the entire EYE-D solution.

3.1 EYE-D System Design (Deployment Architecture)

As illustrated in Figure 4.3, the EYE-D system architecture operates as a dedicated Edge-AI deployment map. The operational flow begins with an Industrial Camera capturing real-time RTSP video streams, which are routed to a local Neural Processing Unit (NPU) for low-latency YOLO inference. Following the on-device processing, the extracted metadata (speed, heading, and hazard alerts) are securely transmitted to the EYE-D Web Dashboard.

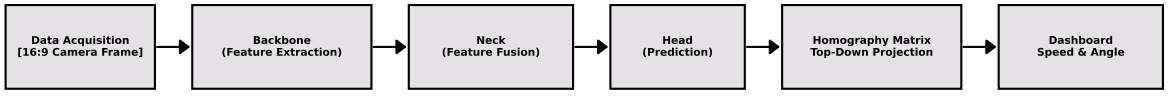


Figure 4.3: EYE-D System Architecture (Edge-AI Deployment Map).

3.2 Integration & Multi-Camera Architectures

To provide a complete engineering blueprint, the system also defines how multiple camera feeds are ingested and processed synchronously.

The UML Integration Architecture specifies the critical interaction between the Video Ingestion module, the NPU Inference Engine, and the Web Dashboard. Specifically, the Video Ingestion module handles the concurrent buffering of multiple RTSP streams, decoding frames directly into NPU-accessible memory space. The NPU Inference Engine then pulls these frames, executing YOLO-based detection and Homography tracking in real time. Finally, the metadata payload-containing timestamps, bounding boxes, object velocities, and alert flags-is serialized and transmitted via WebSockets to the centralized Dashboard, ensuring low-latency situational awareness without overwhelming the network with raw video data.

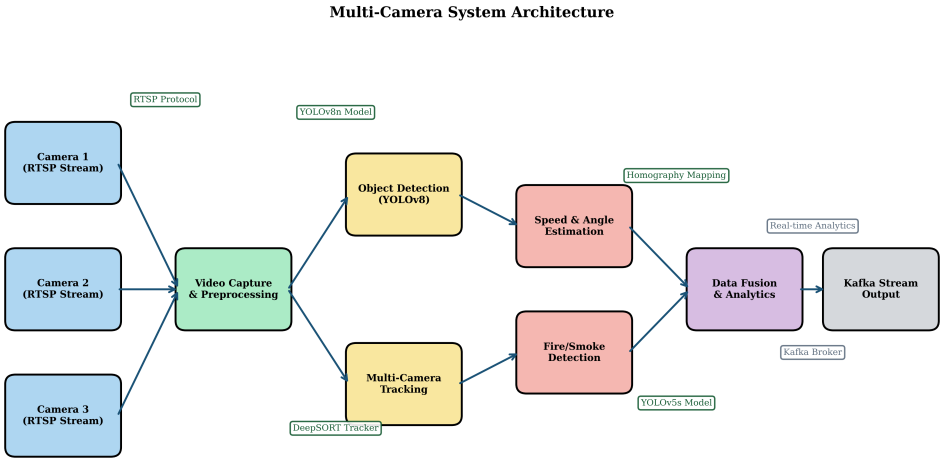


Figure 4.4: Multi-Camera Deployment Architecture for facility-wide monitoring.

3.3 Workflow Pipeline

The velocity estimation logic follows a 10-step fullness verification. First, the Homography matrix **H** projects 2D image coordinates (u, v) to real-world ground coordinates (X, Y). Then, the instantaneous velocity v is calculated using the formula for Euclidean distance over time between consecutive frames:

$$v = \frac{\sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}}{\Delta t} \tag{4.3}$$

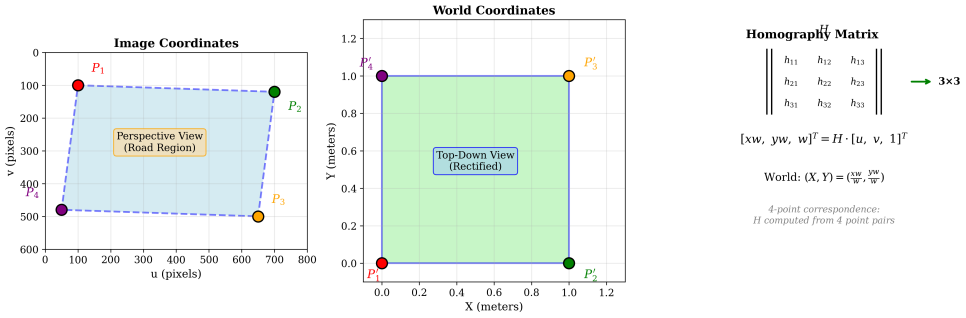


Figure 4.5: Homography-based transformation mapping 2D camera pixels to a top-down ground plane, resolving perspective distortion.

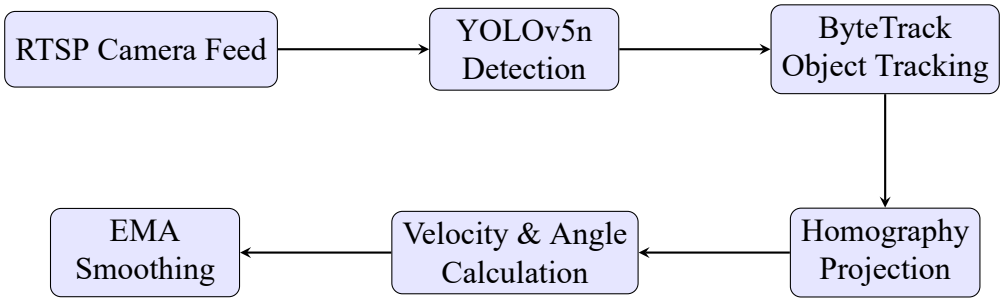


Figure 4.6: Speed estimation horizontal workflow pipeline mapping the sequence from YOLO detection to Homography projection and EMA smoothing.

4 Chapter Summary

This chapter detailed the logistical intelligence capabilities of the system. By leveraging mathematical geometric projections (Homography) and temporal smoothing techniques (Kalman Filters and EMA), the EYE-D system successfully transitions from static hazard detection to dynamic behavioral analysis. The presented system architecture diagrams provide a comprehensive blueprint of how these algorithms are orchestrated on the edge to deliver real-time logistical insights while operating strictly within the SRAM and thermal budgets of the NPU hardware.

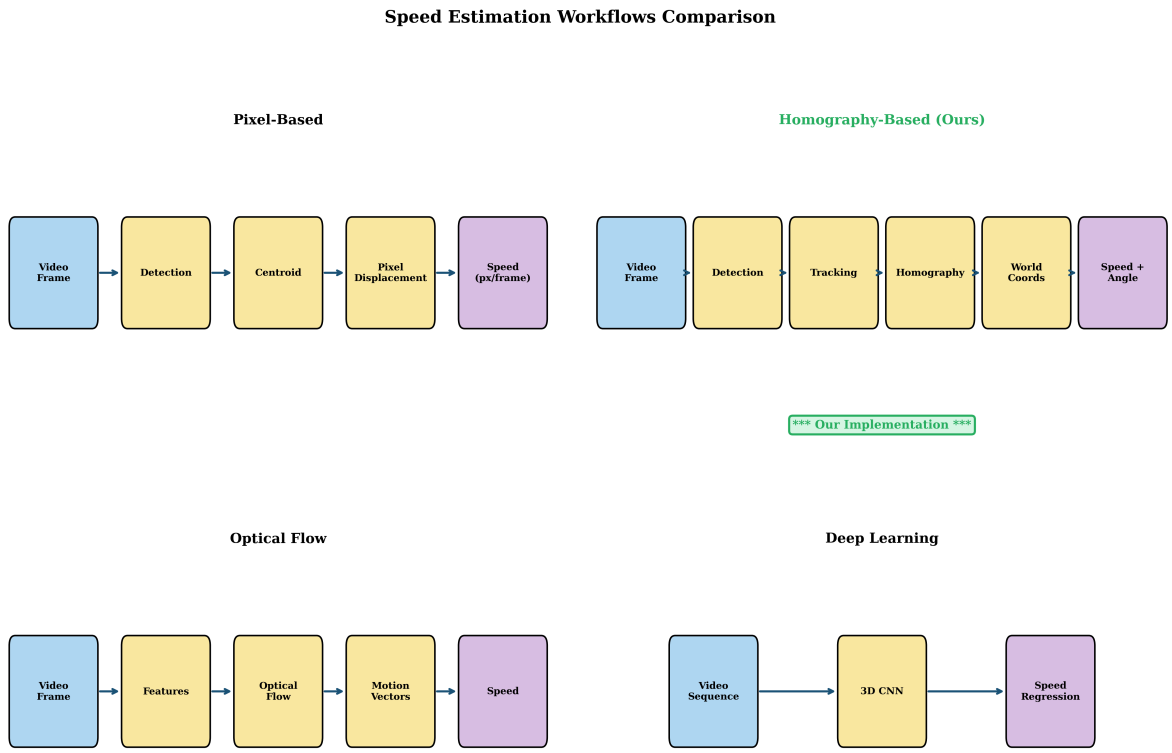


Figure 4.7: Comparison of different speed estimation workflows, highlighting the superiority of Homography for fixed industrial cameras.

Chapter 5

Phase 4 - Zero-Shot Shelf Monitoring

1 Introduction

This chapter presents the technical background and implementation work for the retail inventory capabilities within the EYE-D video analytics solution. The work described here corresponds to Phase 4 of the project, which focused on a Zero-Shot Shelf Fullness Detection system. In contrast to previous supervised learning approaches (e.g., forklift detection), this phase leverages advanced Vision-Language Models (VLMs) to detect objects based on natural language prompts without requiring a single annotated bounding box for that specific class. The primary goal is to make zero-shot models work on specific hardware constraints (NPU) for broader use cases, heavily emphasizing ONNX exportation and optimized inference engines customized for users' specific operational conditions.

2 Zero-Shot Detection Paradigms (SoTA)

2.1 Evolution of Detection: Transition from Bounding Box Regression to Semantic Grounding

The paradigm of computer vision in industrial retail has historically relied on supervised learning, where models are trained to perform bounding box regression on a finite, predefined set of classes. This approach inherently struggles with the dynamic nature of retail environments, where Stock Keeping Units (SKUs) are continuously introduced, removed, or redesigned. Consequently, the field is undergoing a fundamental transition toward semantic grounding. Rather than predicting spatial coordinates tied to an arbitrary integer class ID, semantic grounding aligns visual features with unconstrained textual descriptions. This evolutionary step eliminates the bottleneck of continuous dataset annotation, enabling intelligent systems to generalize to novel objects simply by interpreting natural language queries.

Building upon this foundational shift, the introduction of multi-modal architectures has

bridged the gap between natural language processing and computer vision.

2.2 Vision-Language Models (VLMs)

Vision-Language Models (VLMs) represent the convergence of linguistic comprehension and visual perception. In traditional pipelines, a model only recognizes patterns it has been explicitly trained to see. Conversely, VLMs utilize a dual-encoder architecture where text prompts (e.g., "empty shelf" or "metal shelving") are mapped into dense feature vectors using a language encoder. Simultaneously, the image is processed by a vision encoder to extract spatial features. The model then computes the cross-modal similarity between the text embeddings and the visual regions. This interaction allows the system to identify the precise spatial location of an object based solely on its textual description. This capability, known as **Text-to-Box Grounding**, empowers the system to detect abstract concepts like "negative space" without requiring a single annotated bounding box for that specific class [36].

With the underlying mechanics of VLMs established, it is necessary to examine the specific architectures deployed in this system to balance accuracy and edge constraints.

2.3 Open-Vocabulary Detectors

Open-Vocabulary Detection is the practical application of VLMs, enabling a model to detect any object requested by a user prompt at runtime. To achieve robust performance in edge environments, this system integrates two distinct open-vocabulary architectures: Grounding DINO and YOLO-World. Grounding DINO operates as a high-precision transformer, deeply fusing vision and language features across multiple attention layers to achieve state-of-the-art accuracy in complex semantic queries [36]. However, this precision demands significant computational resources. To mitigate this, YOLO-World is employed as a highly efficient counterpart. YOLO-World leverages a RepVGG-based backbone combined with a streamlined vision-language path aggregation network, optimizing the architecture for real-time inference and lower memory consumption [74].

Table 5.1: Comparative Analysis of Open-Vocabulary Detectors for Edge Deployment

Metric	**Grounding DINO**	**YOLO-World**
Accuracy (Semantic Precision)	Extremely High	High
Latency (Inference Speed)	High	Low (Real-Time)
Memory Usage (Footprint)	High (VRAM intensive)	Low (Optimized for Edge)
Primary Use Case	Complex Text-to-Box Grounding	Resilient, low-latency fallback

This comparative dynamic dictates the operational strategy, ensuring that the highest possible accuracy is maintained without compromising system stability.

2.4 Evaluation Metrics

Evaluating the performance of a zero-shot system requires adapting traditional metrics to an open-vocabulary context. Without pre-defined classes, accuracy must be quantified based on the spatial alignment of the text prompt to the physical reality of the shelf.

To evaluate the zero-shot detector, the standard Mean Average Precision (mAP) is modified to assess semantic similarity thresholds alongside spatial overlap. The zero-shot mAP is defined as:

$$mAP_{ZS} = \frac{1}{N_{prompts}} \sum_{i=1}^{N_{prompts}} \int_0^1 P_i(R) dR \tag{5.1}$$

where $P_i(R)$ is the precision-recall curve for a specific textual prompt i, evaluated over a set of unseen classes or semantic concepts.

Furthermore, the primary operational output is the **Fullness Ratio (%)**, which quantifies the inventory density. Instead of counting individual SKUs, the system leverages "Negative Space Analysis." The ratio is calculated by comparing the detected product area against the total detected empty shelf area:

$$\text{Fullness Ratio (\%)} = \left(\frac{\sum \text{Area}_{products}}{\sum \text{Area}_{products} + \sum \text{Area}_{empty\_voids}} \right) \times 100 \tag{5.2}$$

These mathematical foundations drive the logic of the operational pipeline detailed in the subsequent section.

3 The 10-Step Fullness Pipeline (Design)

3.1 Conceptual Architecture

The conceptual architecture of Phase 3 is anchored in the theory of **Negative Space**. In highly variable retail environments, tracking the presence of every unique product is computationally and logistically prohibitive. Negative Space theory inverts this problem: the system focuses on detecting the structural substrate of the shelf and the explicit absence of products (voids). By quantifying the empty spaces against the known physical boundaries of the shelving unit, the system deduces inventory levels deterministically, rendering it immune to SKU variation.

To execute this conceptual approach reliably on edge hardware, a structured and resilient pipeline is required.

3.2 The Resilient Workflow

Phase A: Validation

1. **Frame Capture:** The industrial camera provides a raw RTSP video frame.

2. **Quality Assessment:** The frame undergoes a rapid check for exposure, blur, and lighting conditions.

3. **Shelf ROI Extraction:** The primary Region of Interest (ROI) containing the shelving unit is isolated.

4. **Perspective Correction:** Homography transformations correct lens distortion, aligning the shelf to a flat 2D plane for accurate spatial calculations.

Phase B: Detection

5. **Prompt Injection:** The system dynamically injects text prompts (e.g., "products", "empty shelf void").

6. **Dual-Polarity Inference:** The VLM executes simultaneously to detect both the positive space (products) and the negative space (voids).

7. **3D Confidence Vector Validation:** Each detection is validated using three distinct metrics:

    - *Classification Score:* The semantic probability that the detected region matches the text prompt.
    - *Detection Probability:* The spatial confidence (Intersection over Union) of the bounding box.
    - *Context Relevance:* A logical check ensuring the detection makes physical sense (e.g., a void cannot overlap entirely with a product).

Phase C: Quantification

8. **Area Aggregation:** The total pixel area of validated products and empty voids is computed.

9. **Fullness % Calculation:** The system calculates the Fullness Ratio using the mathematical formula.

10. **Tier Classification:** The final percentage is mapped to an operational action tier (Tier 1: >80% Optimal, Tier 2: 50-80% Monitor, Tier 3: 20-50% Restock, Tier 4: <20% Critical).

This sequential processing ensures deterministic outputs, but deploying such a pipeline on edge NPUs necessitates strict fault-tolerance mechanisms.

3.3 Fault-Tolerance Logic and NPU Optimization

Operating massive VLMs in an air-gapped, edge-constrained environment poses significant thermal and memory challenges. To ensure uninterrupted operation without cloud connectivity, the system utilizes **INT8 quantization** for Grounding DINO and **FP16 quantization** for YOLO-World via ONNX export [42], drastically reducing their memory footprints.

Despite quantization, sudden spikes in scene complexity can cause Out of Memory (OOM) errors on the NPU. To combat this, the architecture features a **Hot-Swap Fallback Protocol**. An NPU Memory Watchdog continuously monitors VRAM utilization. Under normal conditions, the high-precision Grounding DINO (INT8) acts as the primary inference engine. If the watchdog detects an impending memory threshold breach, it instantly suspends the primary model and routes the image input to the secondary YOLO-World (FP16) engine. This resilient decision-tree logic guarantees that the pipeline never halts, maintaining continuous shelf monitoring under all hardware conditions.

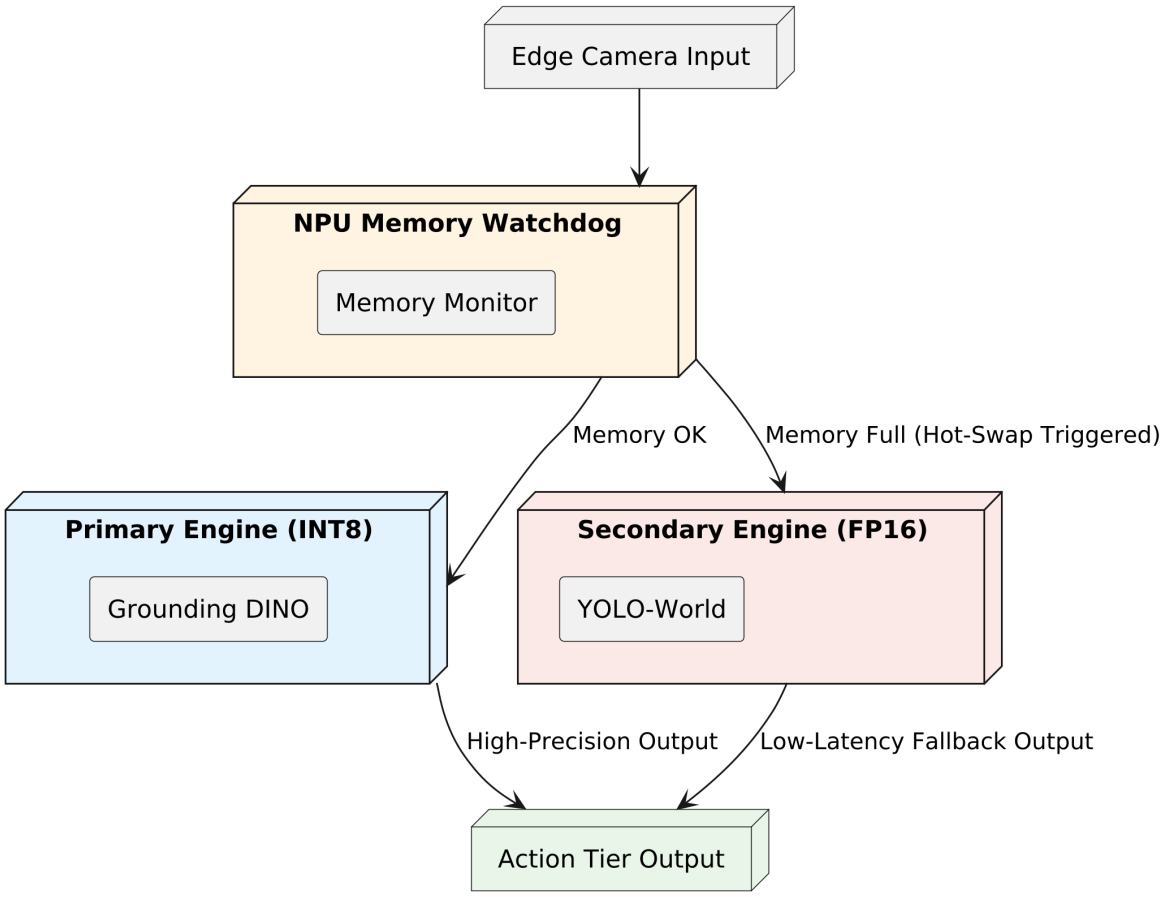


Figure 5.1: Multi-model resilient fallback architecture for NPU-constrained deployment.

4 Implementation Output and Negative Space Detection

The implementation of this zero-shot architecture successfully abstracts away the complexities of SKU-level tracking. Figures 5.2, 5.3, and 5.4 demonstrate the visual output of the zero-shot model detecting shelf voids (negative space) against product space across different inventory levels, verifying the operational success of the models after being exported to ONNX and deployed.

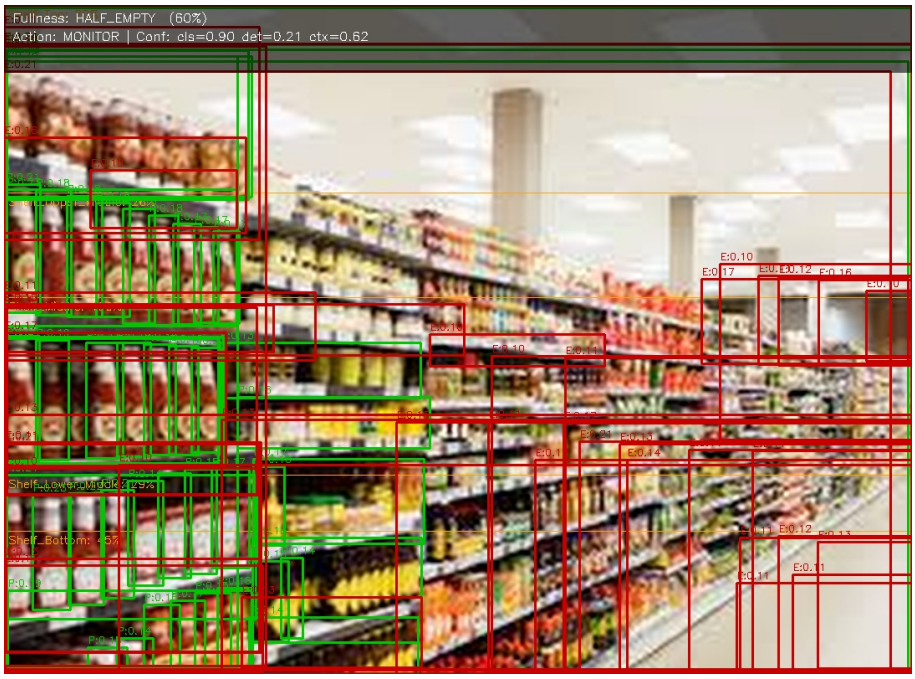


Figure 5.2: Visual example of Negative Space detection identifying shelf voids on a fully stocked shelf.

Figure 5.3: Visual example of Negative Space detection identifying shelf voids on a half-empty shelf.

Figure 5.4: Visual example of Negative Space detection identifying shelf voids on a completely empty shelf.

5 Chapter Summary

By fusing high-accuracy VLMs with robust edge-optimization strategies, the system delivers an air-gapped, fault-tolerant solution capable of real-time inventory monitoring. The resilient fallback mechanism ensures that the theoretical benefits of semantic grounding are practically viable within the strict constraints of industrial NPUs. The most important point of Phase 4 was to verify that the zero-shot model could run effectively on the hardware, proving that clients of the device could easily customize their conditions and dynamically monitor different products using natural language, all without the need for exhaustive data collection or re-training.

Furthermore, this deployment represents the culmination of the project's MLOps lifecycle. To achieve optimal performance on the target NPU, ONNX INT8 quantization was systematically applied to the weights. This process completes the "Full Circle" of the project's engineering methodology: starting from the rigorous data collection and engineering of the 22,607-image dataset (**Phase 1**), progressing through the supervised fine-tuning and hard-negative mining for safety and logistics (**Phases 2 & 3**), and culminating in the NPU optimization and deployment of advanced zero-shot capabilities (**Phase 4**). This comprehensive approach ensures that all models-whether fine-tuned or open-vocabulary-are deeply aligned with the specific hardware constraints and operational realities of the host organization.

General Conclusion

This report serves as a comprehensive study and implementation of an end-to-end Edge-AI computer vision system, developed to run natively on Neural Processing Units (NPUs) for the EYE-D video analytics solution. Our work strategy was inspired by the CRISP-DM methodology, ensuring a structured and iterative approach to overcoming complex industrial challenges.

We began by understanding the business context and the strict hardware requirements, specifically analyzing the severe SRAM memory and compute constraints inherent to edge devices. This allowed us to identify the critical limitations in traditional cloud-centric visual processing workflows and highlight the necessity of migrating inference to the edge. Following this, we established a rigorous data engineering lifecycle, utilizing advanced annotation and analytics tools to curate highly representative datasets for both logistical tracking and industrial safety domains.

Subsequently, we selected, fine-tuned, and optimized single-stage object detection architectures to achieve a precise balance between high-accuracy localization and real-time inference speeds. We addressed critical environmental artifacts and challenging visual conditions, ultimately converting and quantizing the models into NPU-compatible formats. Furthermore, we explored advanced Vision-Language Models and Zero-Shot detection paradigms to evaluate their viability for future, highly adaptable edge deployments.

The achieved results demonstrate the immense potential of Edge-AI in real-world industrial and safety environments. By embedding intelligent processing directly at the edge, our solution ensures rapid responsiveness, strict data privacy, and robust performance, providing a scalable and highly modular foundation for future intelligent video analytics.

Bibliography

[1] P. Chapman *et al.* CRISP-DM 1.0: Step-by-step data mining guide. *SPSS inc*, 2000. (Cited on page 16.)

[2] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. BoT-SORT: Robust Associations Multi-Pedestrian Tracking. *arXiv:2206.14651*, 2022. (Not cited.)

[3] M. Assran *et al.* Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. In *CVPR*, 2023. (Not cited.)

[4] H. Bao, L. Dong, and F. Wei. BEiT: BERT Pre-Training of Image Transformers. In *ICLR*, 2022. (Not cited.)

[5] H. Bilen and A. Vedaldi. Weakly Supervised Deep Detection Networks. In *CVPR*, 2016. (Not cited.)

[6] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao. YOLOv4: Optimal Speed and Accuracy of Object Detection. *arXiv:2004.10934*, 2020. (Not cited.)

[7] A. D. Bodner *et al.* Convolutional Kolmogorov-Arnold Networks. *arXiv:2406.13155*, 2024. (Not cited.)

[8] N. Carion *et al.* End-to-End Object Detection with Transformers (DETR). In *ECCV*, 2020. (Not cited.)

[9] M. Caron *et al.* Emerging Properties in Self-Supervised Vision Transformers (DINO). In *ICCV*, 2021. (Not cited.)

[10] M. Chen *et al.* Generative Pretraining from Pixels (iGPT). In *ICML*, 2020. (Not cited.)

[11] T. Chen *et al.* A Simple Framework for Contrastive Learning of Visual Representations (SimCLR). In *ICML*, 2020. (Not cited.)

[12] Deci AI. YOLO-NAS: A Next-Generation Object Detection Foundation Model. 2023. (Not cited.)

[13] M. Everingham *et al.* The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision*, 2010. (Not cited.)

[14] C. Feng *et al.* TOOD: Task-aligned One-stage Object Detection. In *ICCV*, 2021. (Not cited.)

[15] G. Ghiasi, T.-Y. Lin, and Q. V. Le. NAS-FPN: Learning Scalable Feature Pyramid Architecture for Object Detection. In *CVPR*, 2019. (Not cited.)

[16] R. Girshick. Fast R-CNN. In *ICCV*, 2015. (Not cited.)

[17] R. Girshick *et al.* Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation (R-CNN). In *CVPR*, 2014. (Not cited.)

[18] J.-B. Grill *et al.* Bootstrap Your Own Latent (BYOL). In *NeurIPS*, 2020. (Not cited.)

[19] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*, 2nd Edition. Springer, 2009. (Not cited.)

[20] K. He *et al.* Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2015. (Not cited.)

[21] K. He *et al.* Deep Residual Learning for Image Recognition (ResNet). In *CVPR*, 2016. (Not cited.)

[22] K. He *et al.* Mask R-CNN. In *ICCV*, 2017. (Not cited.)

[23] K. He *et al.* Momentum Contrast for Unsupervised Visual Representation Learning (MoCo). In *CVPR*, 2020. (Not cited.)

[24] K. He *et al.* Masked Autoencoders Are Scalable Vision Learners (MAE). In *CVPR*, 2022. (Not cited.)

[25] G. Huang *et al.* Densely Connected Convolutional Networks (DenseNet). In *CVPR*, 2017. (Not cited.)

[26] G. Jocher *et al.* YOLOv5. GitHub repository. https://github.com/ultralytics/yolov5, 2020. (Not cited.)

[27] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8. GitHub repository. https://github.com/ultralytics/ultralytics, 2024. (Cited on page 21.)

[28] R. Khanam and G. Jocher. YOLO11. Ultralytics, 2024. (Not cited.)

[29] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks (AlexNet). In *NeurIPS*, 2012. (Not cited.)

[30] Y. LeCun, Y. Bengio, and G. Hinton. Deep Learning. *Nature*, 521:436–444, 2015. (Not cited.)

[31] X. Li *et al.* Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection. In *NeurIPS*, 2020. (Not cited.)

[32] T.-Y. Lin *et al.* Microsoft COCO: Common Objects in Context. In *ECCV*, 2014. (Cited on page 10.)

[33] T.-Y. Lin *et al.* Focal Loss for Dense Object Detection (RetinaNet). In *ICCV*, 2017. (Not cited.)

[34] W. Liu *et al.* SSD: Single Shot MultiBox Detector. In *ECCV*, 2016. (Not cited.)

[35] S. Liu *et al.* Path Aggregation Network for Instance Segmentation (PANet). In *CVPR*, 2018. (Not cited.)

[36] S. Liu *et al.* Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. In *ECCV*, 2023. (Cited on page 47.)

[37] Y.-C. Liu *et al.* Unbiased Teacher for Semi-Supervised Object Detection. In *ICLR*, 2021. (Not cited.)

[38] Z. Liu *et al.* KAN: Kolmogorov-Arnold Networks. *arXiv:2404.19756*, 2024. (Not cited.)

[39] J. Long, E. Shelhamer, and T. Darrell. Fully Convolutional Networks for Semantic Segmentation. In *CVPR*, 2015. (Not cited.)

[40] M. Minderer *et al.* Simple Open-Vocabulary Object Detection with Vision Transformers (OWL-ViT). In *ECCV*, 2022. (Not cited.)

[41] Nhathuyy. Forklift Person Dataset. GitHub repository. https://github.com/Nhathuyy/Forklift_Person_Dataset, 2024. (Not cited.)

[42] ONNX. Open Neural Network Exchange. https://onnx.ai/. (Cited on pages 12 and 50.)

[43] OpenCV Documentation. Motion Analysis and Object Tracking. https://docs.opencv.org/4.x/d7/df3/group__imgproc__motion.html. (Not cited.)

[44] A. Radford *et al.* Learning Transferable Visual Models From Natural Language Supervision (CLIP). In *ICML*, 2021. (Not cited.)

[45] J. Redmon and A. Farhadi. YOLOv3: An Incremental Improvement. *arXiv:1804.02767*, 2018. (Not cited.)

[46] J. Redmon *et al.* You Only Look Once: Unified, Real-Time Object Detection (YOLOv1). In *CVPR*, 2016. (Not cited.)

[47] S. Ren *et al.* Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. In *NeurIPS*, 2015. (Not cited.)

[48] R. R. Selvaraju *et al.* Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. In *ICCV*, 2017. (Not cited.)

[49] O. Siméoni *et al.* Localizing Objects with Self-Supervised Transformers and no Labels (LOST). In *BMVC*, 2021. (Not cited.)

[50] O. Siméoni *et al.* Unsupervised Object Localization: Observing and Following (FOUND). In *CVPR*, 2023. (Not cited.)

[51] K. Simonyan and A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition (VGGNet). In *ICLR*, 2015. (Not cited.)

[52] K. Sohn *et al.* A Simple Semi-Supervised Learning Framework for Object Detection (STAC). *arXiv:2005.04757*, 2020. (Not cited.)

[53] M. Tan and Q. V. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *ICML*, 2019. (Cited on page 21.)

[54] M. Tan, R. Pang, and Q. V. Le. EfficientDet: Scalable and Efficient Object Detection. In *CVPR*, 2020. (Not cited.)

[55] J. Tang *et al.* Emergent Correspondence from Image Diffusion. In *NeurIPS*, 2023. (Not cited.)

[56] P. Tang *et al.* Multiple Instance Detection Network with Online Instance Classifier Refinement (OICR). In *CVPR*, 2017. (Not cited.)

[57] J. R. R. Uijlings *et al.* Selective Search for Object Recognition. *International Journal of Computer Vision*, 2013. (Not cited.)

[58] Ultralytics. Speed Estimation with YOLO. https://docs.ultralytics.com/guides/speed-estimation/, 2024. (Not cited.)

[59] C.-Y. Wang *et al.* CSPNet: A New Backbone that can Enhance Learning Capability of CNN. In *CVPR Workshops*, 2020. (Cited on page 11.)

[60] X. Wang *et al.* Cut and Learn for Unsupervised Object Detection and Instance Segmentation (MaskCut). In *CVPR*, 2023. (Not cited.)

[61] Y. Wang *et al.* Self-Supervised Transformers for Unsupervised Object Discovery (TokenCut). In *CVPR*, 2022. (Not cited.)

[62] Y. Wu *et al.* Rethinking Classification and Localization for Object Detection. In *CVPR*, 2020. (Not cited.)

[63] B. Xiao *et al.* Florence-2: Advancing a Unified Representation for a Variety of Vision Tasks. In *CVPR*, 2024. (Not cited.)

[64] M. Xu *et al.* End-to-End Semi-Supervised Object Detection with Soft Teacher. In *ICCV*, 2021. (Not cited.)

[65] Y. Zhang *et al.* ByteTrack: Multi-Object Tracking by Associating Every Detection Box. In *ECCV*, 2022. (Not cited.)

[66] Y. Zhang *et al.* 2D Homography for Traffic Data Collection. *arXiv:2401.07220*, 2024. (Not cited.)

[67] Y. Zhao *et al.* DETRs Beat YOLOs on Real-time Object Detection (RT-DETR). In *CVPR*, 2024. (Not cited.)

[68] B. Zhou *et al.* Learning Deep Features for Discriminative Localization (CAM). In *CVPR*, 2016. (Not cited.)

[69] Q. Zhou *et al.* Dense Teacher: Dense Pseudo-Labels for Semi-supervised Object Detection. In *ECCV*, 2022. (Not cited.)

[70] X. Zhu *et al.* Deformable DETR: Deformable Transformers for End-to-End Object Detection. In *ICLR*, 2021. (Not cited.)

[71] Z. Zong *et al.* DETRs with Collaborative Hybrid Assignments Training (Co-DETR). In *ICCV*, 2023. (Not cited.)

[72] Heartex. Label Studio: Data labeling software. Open source software available from https://github.com/HumanSignal/label-studio, 2024. (Cited on pages 11 and 19.)

[73] Roboflow. Roboflow: Computer Vision Data Management. https://roboflow.com/, 2024. (Cited on pages 11 and 24.)

[74] Cheng, Tianheng, et al. "YOLO-World: Real-Time Open-Vocabulary Object Detection." arXiv preprint arXiv:2401.17270 (2024). (Cited on page 47.)